

A02. Uso de ficheiros

1.A02. Uso de ficheiros	4
1.1	Introdución
1.2	Actividade
1.2.1	Ficheiros de datos
Tipos de acceso	5
Tipos de ficheiros	6
Operacións sobre ficheiros	6
1.2.2	Clase File
1.2.3	Manipulación de ficheiros de texto
Apertura do ficheiro de texto	9
Pecche do ficheiro de texto	10
Lectura de ficheiros de texto	10
Escritura en ficheiros de texto	12
1.2.4	Manipulación de ficheiros binarios
Apertura do ficheiro binario	13
Pecche do ficheiro binario	13
Escritura en ficheiros binarios	13
Lectura de ficheiros binarios	14
1.2.5	Ficheiros de acceso aleatorio
1.2.6	Serialización
1.3	Tarefas
1.3.1	Tarefa 1. Información dun directorio
Enunciado	21
Solución	22
1.3.2	Tarefa 2. Listado de empregados almacenados nun ficheiro ordenados
Enunciado	25
Solución	25
1.3.3	Tarefa 3. Xestión dun ficheiro de empregados
Enunciado	27
Solución	28
1.3.4	Tarefa 4. Copia dun ficheiro binario
Enunciado	32
Solución	32

1.3.5.....	Tarefa 5. Xestión dun ficheiro de acceso aleatorio	35
	<u>Enunciado</u>	35
	<u>Solución</u>	36
1.3.6.....	Tarefa 6. Serialización de obxectos	42
	<u>Enunciado</u>	42
	<u>Solución</u>	42
1.3.7.....	Tarefa 7. Lectura e modificación de obxectos almacenados nun ficheiro	45
	<u>Enunciado</u>	45
	<u>Solución</u>	45

2.Autoavaliación.....51

2.1.....	<u>Enunciado</u>	51
	<u>Exemplo de execución</u>	52

1. Uso de ficheiros

1.1 Introducción

Na actividade que nos ocupa aprenderanse os seguintes conceptos e manexo de destrezas:

- Establecer a estrutura correcta para almacenar información nun ficheiro.
- Diferenciar os distintos tipos de ficheiros.
- Representar a un ficheiro mediante unha ruta correcta.
- Acceder ao contido dun directorio do sistema operativo.
- Crear ficheiros no sistema operativo.
- Borrar ficheiros no sistema operativo.
- Establecer a orde correcta das operacións a realizar nun ficheiro para a súa manipulación.
- Almacenar información nun ficheiro de texto.
- Recuperar información dun ficheiro de texto.
- Almacenar información nun ficheiro binario.
- Recuperar información dun ficheiro binario.
- Almacenar información nun ficheiro de acceso aleatorio.
- Recuperar información dun ficheiro de acceso aleatorio.
- Serializar obxectos.
- Recuperar obxectos dun ficheiro de obxectos.

1.2 Actividade

1.2.1 Ficheiros de datos

Ata este momento, as estruturas de datos utilizadas para o almacenamento de información residen en memoria. Isto ten dúas limitacións importantes:

- Os datos non se almacenan de forma permanente. Unha vez que se sae do programa e volvemos a executalo non podemos acceder á información xerada na execución anterior do programa.
- A cantidade dos datos non pode ser moi grande debido á limitación do tamaño da memoria.

Para evitar estes inconvenientes existen unhas estruturas que utilizan a memoria secundaria para o almacenamento da información as cales se identifican baixo un nome e unha extensión. Estas estruturas denomínanse ficheiros de datos e podemos definilos coma un conxunto de datos, sobre un mesmo tema, tratado coma unha unidade de almacenamento e organizado de forma estruturada para a procura dun dato individual.

Podemos considerar ao ficheiro como unha estrutura dinámica no senso de que o seu tamaño pode variar durante a execución do programa dependendo da cantidade de datos que teña.

Como xa se comentou, a información, ou datos do ficheiro, atópase almacenada seguindo unha determinada estrutura. A esta estrutura se lle denomina rexistro.

Podemos definir un rexistro coma un conxunto de datos que forman unha unidade de información e que se manipulan coma un bloque. Cada rexistro está composto de unidades elementais de información máis pequenas que se denominan campos.

Na seguinte imaxe vemos un exemplo dun ficheiro que almacena datos de empregados. Cada unha das liñas do ficheiro correspóndese cun rexistro, que á súa vez componse de varios campos, onde cada un fai referencia á seguinte información: nome, dni, idade, departamento.

Nome	DNI	Idade	Departamento
Ficheiro: empregados.dat			
Pedro	11111111-A	25	RR.HH.
Ana	22222222-B	32	Programación
Xan	33333333-C	50	Análise&Deseño
Eduardo	44444444-D	29	Administración
Francisco	55555555-E	33	Recepción
Marta	66666666-F	35	Análise&Deseño
Sofía	77777777-G	47	RR.HH

Tipos de acceso

Como o ficheiro está almacenado en memoria secundaria non é posible acceder a toda a súa información de forma inmediata, senón que de cada vez se pode acceder a un rexistro do ficheiro.

Dependendo de como se acceda a cada un destes rexistros podemos distinguir dous tipos de acceso:

- Acceso secuencial: para ler o ficheiro debe comezarse sempre desde o primeiro rexistro, e logo a lectura avanza cara adiante lendo os rexistros dun en un.
- Acceso aleatorio: podemos acceder directamente a calquera posición do ficheiro, e por tanto ler calquera rexistro.

Tipos de ficheiros

A clasificación por tipo de ficheiro baséase na forma na que se garda a información dentro do ficheiro. Existen dous tipos principais de ficheiros:

- Ficheiros de texto: a información almacénase en formato de caracteres, tal e como se vería por pantalla. Por exemplo, o valor enteiro 25 transformarase nos caracteres '2' e '5' que serán almacenados no ficheiro.
- Ficheiros binarios: as secuencias de elementos almacenadas no ficheiro son dun determinado tipo de datos. É dicir, a información gárdase no ficheiro tal e como está en memoria principal, polo tanto, estará composta por uns e ceros. Por exemplo, o valor enteiro 25 gardarase como a secuencia binaria 00011001.

Outra clasificación que se pode facer sobre os ficheiros é a que distingue entre:

- Ficheiros físicos: é o ficheiro que reconece o sistema operativo e que está almacenado en memoria secundaria baixo un nome e unha extensión.
- Ficheiros lóxicos: o programa debe poder facer referencia ao ficheiro físico para traballar con el. Desta maneira as linguaxes de programación proporcionan mecanismos para poder crear un tipo de dato de tipo ficheiro. En Java, se queremos crear unha variable que apunta a un ficheiro físico debemos utilizar a clase `File`. Desta forma o ficheiro lóxico será un obxecto da clase `File`.

Tamén podemos clasificar os ficheiros segundo a forma na que se abren para proceder á súa manipulación. Existen dous tipos:

- Ficheiros de lectura: son aqueles que se abren e sobre os que só se pode ler a información que conteñen. Non se poderá facer ningunha modificación no seu contido.
- Ficheiros de escritura: son aqueles ficheiros que ademais de poder ler os datos que conteñen, permiten a modificación do seu contido. Por exemplo, poderían engadirse novos rexistros.

Operacións sobre ficheiros

As operacións que se deben realizar sobre un ficheiro para poder traballar con el son as seguintes:

- Apertura do ficheiro: consiste en crear a asociación entre o ficheiro lóxico e o ficheiro físico. Para realizar esta operación, é necesario coñecer, ademais do nome e extensión do ficheiro, a ruta en memoria secundaria onde este estea almacenado. Despois de aberto o ficheiro xa é posible empezar a traballar con el dentro da aplicación. Cando se realiza esta operación hai que ter en conta que o ficheiro para poder ser aberto debe existir, xa que senón se vai a xerar un erro. Ademais, neste momento é cando se decide se o ficheiro se vai a abrir só para lectura ou tamén para escritura.

- Operacións de manipulación: segundo a forma na que se abra o ficheiro pódense realizar as seguintes dúas operacións:
 - Lectura do ficheiro: permite recuperar a información gardada no ficheiro e poñela en memoria para poder traballar con ela. A maneira de ler o ficheiro dependerá do tipo no que estea gardada a información, que pode ser ben de tipo texto ou de tipo binaria.
 - Escritura en ficheiro: esta operación permite poñer información que está na memoria no ficheiro, para poder utilizala posteriormente. Ao igual ca na lectura, a maneira de escribir no ficheiro dependerá do tipo no que queiramos gardar a información, ben de tipo texto ou de tipo binaria.
- Pechado do ficheiro: cando se acabe de realizar todas as operacións necesarias sobre o ficheiro, este debe pechase sempre. No caso de Java, o peche permite liberar o fluxo de datos creado na apertura do ficheiro.

Antes de comezar a traballar cos ficheiros e poder manipular a información que conteñen tal como inserir novos datos no ficheiro, facer algunha modificación neles ou incluso borrarlos, vamos a ver como traballar co ficheiro físico que está almacenado na memoria secundaria.

1.2.2 Clase File

A clase `File`, ao igual que outras clases relacionadas coa entrada e saída de información en Java, atópase no paquete `java.io`. Esta clase permite realizar operacións sobre o ficheiro a nivel do sistema de arquivos: tal como listar os arquivos, crear directorios, borrar ficheiros, modificar o seu nome, etc.

Os sistemas operativos utilizan un nome de ruta para facer referencia aos arquivos e directorios que dependen do propio sistema operativo. Esta clase permite obter unha representación abstracta da ruta que é independente do sistema.

Este nome de ruta pode ser de dous tipos:

- Ruta absoluta: neste caso a ruta contén a información completa para poder acceder ao arquivo. Polo tanto, nesta ruta debe indicarse antes do ficheiro o directorio raíz. Podemos dicir que a ruta absoluta está formada por un prefixo, un listado de directorios e o nome do directorio ou arquivo ao que queremos acceder.
- No caso do sistema operativo Windows, o prefixo está composto polo nome da unidade, seguido do carácter “:” e a continuación ou ben “/” ou “\”. O listado de directorios para chegar ao destino sepáranse ben con “/” ou “\”. A continuación móstrase a ruta absoluta para chegar ao arquivo `arquivo.txt`.

Ruta absoluta Windows

```
C:\directorio1\directorio11\arquivo.txt
```

- No caso do sistema operativo Unix, o prefixo está composto sempre por “/” que representa a raíz do sistema de directorios. O listado de directorios sepáranse polo carácter “/”: A continuación móstrase a ruta absoluta para chegar ao arquivo `arquivo.txt`.

Ruta absoluta Unix

```
/home/user1/directorio1/directorio11/arquivo.txt
```

- Ruta relativa: neste caso só é necesario indicar o nome do directorio ou arquivo ao cal queremos acceder. Usando a ruta relativa, o arquivo indicado procúrase dentro do directorio actual, que é o directorio do arquivo Java no que se está traballando.

Para crear un obxecto de tipo arquivo a partir do sistema de ficheiros do sistema operativo, a clase `File` proporciona 3 construtores:

- Un construtor que ten un só argumento de tipo cadea que representa a ruta ao sistema de arquivos.

Creación de ficheiros

```
//Ruta absoluta Windows
File arquivoW = new File("C:/info/archivos/archivo.txt");
//Ruta absoluta Unix
File arquivoU = new File("/home/info/archivos/archivo.txt");
//Ruta relativa
File arquivo = new File("archivo.txt");
```

- Dous construtores nos que se deben especificar dous argumentos:

- O primeiro argumento pode ser de tipo `File` ou unha cadea de caracteres. En ambos casos representa á ruta pai.
- O segundo argumento é de tipo cadea e representa á ruta filla. Esta ruta filla é unha segunda ruta que se define a partir da posición da primeira ruta, da ruta pai.

Creación de ficheiros con dous argumentos

```
File directorio = new File("C:/info");
File arquivo = new File(directorio,"/archivos/archivo.txt");
```

Cando se crea un arquivo hai que ter en conta que non se devolve unha excepción cando o arquivo non existe. A creación dun obxecto de tipo arquivo só devolve unha excepción, a excepción `NullPointerException`, cando a ruta completa (no caso do construtor dun só argumento) ou a ruta filla (no caso do construtor de dous argumentos), son nulas. É por iso, que en conxunto coa creación do obxecto de tipo arquivo débese utilizar o método da clase `exists()` que devolve `true` se o directorio ou arquivo é válido ou `false` en caso contrario.

A Máquina Virtual de Java permítenos executar código escrito en Java independentemente da plataforma desde o que se realiza a execución. Desta forma, en Java podemos escribir programas que se poden executar en calquera plataforma. Esta grande versatilidade de Java a debemos ter en conta cando especificamos a ruta do arquivo. Dado que cando se crea o programa non se sabe en que plataforma se vai executar, non deberíamos utilizar nomes de ruta que dependan tan directamente do sistema operativo a utilizar.

Para evitar este problema debemos usar o separador “/” que é válido tanto para Windows coma para Linux ou tamén podemos facer uso das variables estáticas que posúe `File`.

Variable estática	Descrición
<code>public static final char separatorChar</code>	Representa ao carácter separador de nomes de arquivos e carpetas.
<code>public static final String separator</code>	Como o anterior pero en forma de String.

Uso do separador

```
String ruta="UsersPublic/Documents/datos.txt";
ruta=ruta.replace("/", File.separator);
```

Podemos clasificar toda a lista de métodos da clase `File` do seguinte xeito:

- Métodos xerais: que proporcionan información xeral sobre o obxecto. Permítenos saber entre outras cousas se o ficheiro existe, se se pode ler, se está oculto ou se dous arquivos son iguais.
- Métodos de directorio: permiten facer operacións sobre un directorio. Por exemplo, podemos crear un directorio dentro del ou obter a lista dos arquivos que contén.

- Métodos de arquivo: permiten facer operacións sobre un arquivo. Entre outros métodos, hai métodos para renomear un arquivo, para borrarlo ou para saber a súa lonxitude.

Deberanse consultar todos os métodos da clase `File` antes de comezar a realizar a tarefa



Enlace á API de Java: <https://docs.oracle.com/javase/8/docs/api/>



Realizar a Tarefa 1 “Información dun directorio”, onde o alumnado deberá ler de teclado un directorio e deberá permitir que se realicen unha serie de operacións sobre el.

1.2.3 Manipulación de ficheiros de texto

A manipulación de ficheiros de texto pode facerse desde as clases de fluxo de bytes `FileInputStream` ou `FileOutputStream`.

Nembargantes, cando se trata de traballar especificamente con arquivos de texto é mellor transformar os fluxos anteriores ou ben a `Reader` ou `Writer`, segundo corresponda, mediante as clases `InputStreamReader` e `OutputStreamReader`. Senón tamén está a opción ideal que é traballar directamente coas clases `FileReader` e `FileWriter`, para facer a lectura ou a escritura do ficheiro de texto respectivamente. Nós vamos a utilizar esta última opción.

A continuación vamos a ir vendo cada unha das operacións que debemos realizar para a manipulación de ficheiros, neste caso adaptadas aos ficheiros de texto.

Apertura do ficheiro de texto

A primeira operación que temos que realizar é a de apertura do ficheiro. Neste paso hai que decidir se queremos abrir o ficheiro só para lectura o tamén para escritura.

Se queremos abrir o ficheiro só para lectura utilizamos a clase `FileReader` que dispón dos seguintes construtores:

Construtores ficheiro de lectura

```
FileReader(File arquivo);  
FileReader(String nomeArquivo);
```

Como se pode comprobar nas dúas sentenzas anteriores a construción dun obxecto pode facerse ou ben cun parámetro que pode ser de tipo `File` ou un `String`. De ámbolos dous xeitos se pode representar a un determinado arquivo.

Se quixeramos abrir o ficheiro tamén para escritura deberemos utilizar neste caso a clase `FileWriter`. Os construtores dispoñibles son os mesmos que na clase `FileReader`, e permiten abrir o ficheiro directamente para escritura. Esta clase ademais incorpora outros dous construtores cun parámetro extra de tipo booleano, que en caso de valer `true`, indica que se abre o arquivo para engadir datos, en caso contrario, abrírase para gravar desde cero co que borrará o contido previo que tivese o arquivo.

Construtores ficheiro de escritura

```
FileWriter(File arquivo);  
FileWriter (String nomeArquivo);
```

```
FileWriter (File arquivo, boolean engadir);  
FileWriter (String nomeArquivo, boolean engadir);
```

Cando se abre o ficheiro un cursor apunta ao inicio do ficheiro. Este cursor indica a seguinte información do ficheiro á que se pode acceder.

A apertura do fluxo cara ao ficheiro produce unha excepción que hai que controlar. Esta excepción prodúcese cando o ficheiro non existe, cando é un directorio en lugar dun ficheiro ou por calquera outra razón que impida abrir o ficheiro. Esta excepción pode ser de dous tipos:

- `FileNotFoundException` cando se usan os métodos de `FileReader`.
- `IOException` cando se usan os métodos de `FileWriter`.

Peché do ficheiro de texto

Unha vez que se remate de utilizar o ficheiro debe usarse o método `close` que pecha o fluxo e ademais libera calquera recurso que sexa utilizado polo fluxo. É aconsellable incluír este método dentro do bloque `finally` de forma que sempre se execute.

Exemplo de peche do ficheiro

```
try {  
    FileReader ficheiro = new FileReader("datos.dat")  
    //instrucións  
} catch (IOException e) {  
    //instrucións  
} finally {  
    ficheiro.close();  
}
```

Desde a versión 7 de Java pode utilizarse tamén a instrución `try-with-resources`. Esta sentenza declara un ou máis recursos, considerando coma tal un obxecto que debe ser pechado despois de que o programa remate de utilizalo. Desta forma, podémonos asegurar que cada recurso será liberado unha vez que finalice a sentenza.

A continuación, pódese ver un exemplo onde se abre un ficheiro para a súa lectura usando a instrución `try-with-resources`. Unha vez que se executen todas as instrucións dentro do bloque `try` pecharase o fluxo declarado ao seu comezo.

Exemplo de uso `try-with-resources`

```
try (FileReader ficheiro = new FileReader("datos.dat")) {  
    //instrucións  
} catch (IOException e) {  
    //instrucións  
}
```

Lectura de ficheiros de texto

Para realizar a lectura pódese utilizar o método `read()` que pode recibir un array de caracteres onde se almacenarán os caracteres lidos. Este método devolve un valor de tipo enteiro que indica o número de caracteres lidos, ou ben terá valor -1 para indicar que se chegou ao final da lectura.

Este método é un pouco rudimentario polo que o ideal é converter o fluxo nunha clase de tipo `BufferedReader`. Esta clase proporciona os seguintes métodos de lectura:

Método	Descrición
<code>int read()</code>	Le un carácter de forma individual e o devolve coma un enteiro que estará dentro do rango 0...65535. Devolverá -1 cando chegue ao final do fluxo.
<code>int read(char[] buffer, int off, int lonxitude)</code>	Almacena nun array de caracteres o que se le. Débese indicar unha posición desde a que comezará a lectura e o número máximo de caracteres a ler. Devolve un enteiro que

	será o número de caracteres lidos ou -1 cando chegue ao final do fluxo. Devolverá null cando se alcance o final do fluxo.
String readLine()	Le unha liña completa de texto. Considerarase que a liña rematou cando se atope un salto de liña '\n', un retorno de carro '\r' ou un retorno de carro seguido inmediatamente por un salto de liña.

Estes métodos lanzan unha excepción de tipo `IOException`, que se deberá ter en conta á hora de utilizalos.

Cada vez que se executa o método `read()` o cursor avanza á seguinte posición. En caso de estar lendo de carácter en carácter o cursor avanzará ao seguinte carácter, se se le a liña completa entón o cursor avanzará á seguinte liña.

Vamos a ver un exemplo de como ler o contido dun ficheiro carácter a carácter. Para elo consideremos que no directorio do proxecto temos un ficheiro chamados `datos.dat` co seguinte contido:

```
4    6
5    6
7    1
43   5
65   77
```

Exemplo lectura ficheiro carácter a carácter

```
String contido = "";
int dato;
try {
    //Apertura do fluxo de lectura cara ao ficheiro
    //O cursor sitúase ao comezo do ficheiro
    FileReader ficheiro = new FileReader("datos.dat");
    //Lectura do primeiro carácter
    //O cursor móvese cara ao seguinte carácter
    dato = ficheiro.read();
    //Mentres o valor que retorne o método de lectura non sexa -1
    //significa que aínda hai máis caracteres por ler no ficheiro
    while (dato != -1) {
        //O valor do método de lectura devolve un enteiro que debe
        //ser convertido correctamente para a súa visualización
        contido = contido + (char) dato;
        dato = ficheiro.read();
    }
    System.out.println("O contido do ficheiro é o seguinte:\n" + contido);
    //Peche do fluxo e liberación de recursos
    ficheiro.close();
    //Pode evitarse o primeiro catch, posto que FileNotFoundException é unha
    //excepción filla de IOException. Desta maneira podemos diferenciar os erros
} catch (FileNotFoundException e) {
    System.out.println("ERRO: O ficheiro é un directorio ou ben non existe.");
} catch (IOException e) {
    System.out.println("ERRO: Ocorreu un erro na lectura do ficheiro");
}
```



Realizar a Tarefa 2 “Listado ordenado de empregados almacenados nun ficheiro”, onde o alumnado deberá ler o contido dun ficheiro, cargar a información en memoria e mostrar a información ordenada.

Escritura en ficheiros de texto

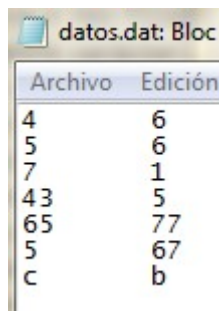
Para poder escribir no ficheiro, previamente teremos que abrílo para escritura como se viu anteriormente. Unha vez que se pode escribir no ficheiro podemos utilizar o método `write()`, que permite escribir tanto un único carácter, coma un `String` ou un array de bytes.

Ao igual que ocorría na lectura de ficheiros, se utilizamos un buffer para escribir en ficheiros, coa clase `BufferedWriter`, podemos utilizar ademais o método `newLine()` que vai escribir un salto de liña no arquivo. Co uso deste método evitamos o problema da compatibilidade entre plataformas, xa que os caracteres de cambio de parágrafo son distintos segundo cada sistema operativo.

Exemplo de escritura de ficheiro

```
String contido = "";
int dato;
try {
    //Apertura do fluxo de lectura cara ao ficheiro
    //O cursor sitúase ao comezo do ficheiro
    FileWriter ficheiro = new FileWriter("datos.dat", true);
    //Lectura do primeiro carácter
    //O cursor móvese cara ao seguinte carácter
    contido = "5\t67\r\n";
    //Escritura no ficheiro usando unha cadea
    ficheiro.write(contido, 0, contido.length());
    //escritura no ficheiro usando caracteres
    ficheiro.write((int) 'c');
    ficheiro.write((int) '\t');
    ficheiro.write((int) 'b');
    ficheiro.write((int) '\r');
    ficheiro.write((int) '\n');
    //Peché do fluxo e liberación de recursos
    ficheiro.close();
} catch (IOException e) {
    System.out.println("ERRO: Ocorreu un erro na escritura do ficheiro");
}
```

Tendo en conta que o arquivo é o mesmo que no exemplo anterior, agora o arquivo conterá a seguinte información:



Archivo	Edición
4	6
5	6
7	1
43	5
65	77
5	67
c	b



Realizar a Tarefa 3 “Xestión dun ficheiro de empregados”, onde o alumnado deberá, a partir da aplicación da Tarefa 2, ler o contido dun ficheiro que contén os datos dos empregados dunha empresa, traballar cos datos lidos, engadir novos empregados e escribir datos no ficheiro.

1.2.4 Manipulación de ficheiros binarios

Un ficheiro binario está formado por secuencia de bytes. Dependendo de se coñecemos a estrutura interna do ficheiro podemos ler o seu contido dunha forma ou doutra. Se non coñecemos a estrutura entón debemos ler o ficheiro byte a byte. En caso de coñecer a es-

trutura e de saber o tipo (int, float, char, etc) e a orde dos datos podemos utilizar métodos máis específicos para a lectura de cada tipo de dato.

Apertura do ficheiro binario

A clase que nos permite abrir un ficheiro binario para a súa lectura é `FileInputStream` e ten os seguintes construtores:

Construtores para abrir ficheiro binario en modo de lectura

```
FileInputStream (String ruta)
FileInputStream (File objetoFile);
```

A clase que nos permite abrir un ficheiro binario para a súa escritura é `FileOutputStream` e ten os seguintes construtores:

Construtores para abrir ficheiro binario en modo de escritura

```
FileOutputStream (String ruta)
FileOutputStream (File objetoFile);
FileOutputStream (String ruta, boolean append)
FileOutputStream (File objetoFile, boolean append)
```

Pódese comprobar que son idénticos que no caso dos ficheiros de texto, polo que as dúas primeiras opcións abren o ficheiro para escritura de maneira que os datos existentes se perderán. Se se empregan as últimas dúas opcións e se o parámetro booleano está a `true`, entón os datos anteriores permanecen e os novos engádiranse ao final do ficheiro, en caso de que sexa `false`, borraranse os datos existentes antes de proceder coa escritura.

Todos os construtores mencionados neste apartado, lanzan unha excepción `FileNotFoundException` cando o ficheiro non existe ou se non se puido crear o ficheiro en caso de abrir o ficheiro para escritura.

Peché do ficheiro binario

Ao igual que ocorre cos ficheiros de texto, os fluxos de datos binarios deben ser pechados para liberar calquera recurso que estivese utilizando. Para elo, existe o mesmo método que no caso de ficheiros de texto:

Método de peche de ficheiros

```
close();
```

Se hai un erro de entrada/saída, este método lanza unha excepción de tipo `IOException`.

É aconsellable incluír esta instrución de peche dentro do bloque `finally` para que sempre se execute.

Pode evitarse ter que estar pendente do peche do fluxo declarando o fluxo nunha instrución `try-with-resources` para que sexa o propio programa o que se encargue do peche do fluxo ao final da execución da instrución.

Escritura en ficheiros binarios

A clase utilizada para abrir un fluxo de bytes en modo escritura, `FileOutputStream`, proporciona o método `write()` para escribir bytes no ficheiro. Cando usemos este método hai que ter en conta que lanza unha excepción de tipo `IOException`.

Exemplo de escritura en ficheiro binario

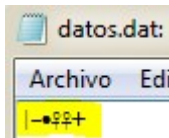
```
FileOutputStream ficheiro = null;
```

```

try {
    ficheiro = new FileOutputStream("datos.dat");
    byte[] datos = {5, 6, 7, 12, 12, 43};
    ficheiro.write(datos);
} catch (IOException e) {
    System.out.println("ERRO: Ocorreu un erro na escritura do ficheiro");
} finally {
    try {
        //Comprobamos se o fluxo está aberto.
        //Se non está aberto non é necesario pechalo
        if (ficheiro != null) {
            ficheiro.close();
        }
    } catch (IOException e) {
        System.out.println("ERRO: Ocorreu un erro no peche do ficheiro");
    }
}

```

O resultado da execución do código anterior é a creación do arquivo datos.dat coa seguinte información:



Se quixeramos escribir no ficheiro datos de tipo primitivo, como por exemplo datos de tipo `int`, podemos crear un obxecto `DataOutputStream` a partir do obxecto `FileOutputStream`, xa que esta clase proporciona métodos específicos para a escritura de datos de tipo primitivo no ficheiro.

Para crear un obxecto de tipo `DataOutputStream` utilízase o seguinte construtor:

Construtor

```
DataOutputStream(OutputStream nome);
```

Os métodos para escribir os datos de tipo primitivo teñen a seguinte forma:

Método de escritura tipo primitivo

```
writeXxx(); //Onde Xxx é o nome do tipo primitivo.
```



Acceder ao enlace da API de Java para consultar todos os métodos de lectura da clase `DataOutputStream`: <https://docs.oracle.com/javase/8/docs/api/>

Ao igual que o método xeral para escribir bytes, estes métodos tamén lanzan a excepción `IOException`.

Lectura de ficheiros binarios

A clase utilizada para abrir un fluxo de bytes en modo lectura, `FileInputStream`, proporciona o método `read()` para ler bytes desde o ficheiro. Cando usemos este método hai que ter en conta que lanza unha excepción de tipo `IOException`.

Se quixeramos ler do ficheiro datos de tipo primitivo, como por exemplo datos de tipo `int`, podemos crear un obxecto `DataInputStream` a partir do obxecto `FileInputStream`, xa que esta clase proporciona métodos específicos para a lectura de datos de tipo primitivo no ficheiro.

No seguinte exemplo pode comprobarse como se faría a lectura do ficheiro creado no apartado anterior:

Exemplo de lectura en ficheiro binario

```

FileInputStream ficheiro = null;
try {
    ficheiro = new FileInputStream("datos.dat");
    byte[] datos = new byte[ficheiro.available()];
    int valor = ficheiro.read(datos);
    if (valor != 0){
        for (int i = 0; i < valor; i++) {
            System.out.print(datos[i]+"\\t");
        }
    }
} catch (IOException e) {
    System.out.println("ERRO: Ocorreu un erro na escritura do ficheiro");
} finally {
    try {
        //Comprobamos se o fluxo está aberto.
        //Se non está aberto non é necesario pechalo
        if (ficheiro != null) {
            ficheiro.close();
        }
    } catch (IOException e) {
        System.out.println("ERRO: Ocorreu un erro no peche do ficheiro");
    }
}
}

```

Para crear un obxecto de tipo `DataInputStream` utilízase o seguinte construtor:

Construtor

`DataInputStream(InputStream nome);`

Os métodos para ler os datos de tipo primitivo teñen a seguinte forma:

Método de lectura dato primitivo

`readXxx();` //Onde `Xxx` é o nome do tipo primitivo.



Acceder ao enlace da API de Java para consultar todos os métodos de lectura da clase `DataInputStream`: <https://docs.oracle.com/javase/8/docs/api/>

Ao igual que o método xeral para escribir bytes, estes métodos tamén lanzan a excepción `IOException`.

Cando un método `readXxx()` alcanza o final do ficheiro lanza unha excepción `EOFException`.



Realizar a Tarefa 4 “Copia dun ficheiro binario”, onde o alumnado deberá crear un ficheiro binario a partir dos datos introducidos polo usuario e ademais deberá facer unha copia deste ficheiro e mostrar os datos por pantalla.

1.2.5 Ficheiros de acceso aleatorio

Os tipos de ficheiros vistos ata agora son secuenciais, o que quere dicir que para acceder a un determinado rexistro débese recorrer secuencialmente desde o inicio do ficheiro e pasar por todos os rexistros ata atopar o rexistro que nos interese.

Neste apartado vamos a falar de como traballar cos ficheiros de acceso aleatorio. Este tipo de ficheiros ten as seguintes características:

- Non se trata dun fluxo de datos como nos ficheiros de texto ou binarios.
- Almacena un conxunto de bytes.

- Permite o acceso aleatorio a un determinado rexistro, desta forma non é necesario recorrer os rexistros anteriores para acceder a un rexistro concreto.
- Pódese abrir para lectura ou para lectura e escritura.
- Ten un cursor ou índice chamado file pointer (nos usaremos o termo punteiro), que sinala cara un punto do ficheiro. Inicialmente apunta ao comezo do ficheiro e pódese mover á outra posición diferente. As operacións de lectura comezan a ler bytes a partir da posición á que apunta o punteiro nese instante e o desprazan tantos bytes como foron lidos. No caso das operacións de escritura, os bytes empézanse a escribir a partir da posición indicada polo punteiro e este avanza ata pasados os bytes que foron escritos.
- Dispón de operacións de lectura e escritura baseadas no tipo de dato a ler ou a escribir no ficheiro.

A clase que permite o uso de ficheiros de acceso aleatorio é a clase `RandomAccessFile` que tamén se atopa no paquete `java.io`. Esta clase proporciona dous construtores:

Construtores

```
RandomAccessFile(File arquivo, String modo);
RandomAccessFile(String nome, String modo);
```

Os dous construtores lanzan a excepción `FileNotFoundException` que hai que tratar.

A forma na que indicamos o arquivo que queremos abrir en modo de acceso aleatorio o podemos facer creando un obxecto de tipo `File` o ben indicando a cadea de texto que corresponde co ficheiro.

Ambos construtores teñen un parámetro obrigatorio que indica o modo no que se abrirá o ficheiro. Os valores permitidos para este parámetro son as que se indican na seguinte táboa:

Modo	Descrición
"r"	Abre o ficheiro para só lectura. Se utilizamos algún dos métodos de escritura sobre o obxecto creado lanzarase unha excepción <code>IOException</code> .
"rw"	Abre o ficheiro para lectura e escritura. Se o ficheiro non existe entón intentará crealo.
"rws"	Abre o ficheiro para lectura e escritura e ademais require que cada actualización do contido ou metadatos do arquivo se escriba de forma sincronizada no dispositivo de almacenamento subxacente.
"rwd"	Abre o ficheiro para lectura e escritura e ademais require que cada actualización do contido do arquivo se escriba de forma sincronizada no dispositivo de almacenamento subxacente.

Os métodos xerais de lectura e escritura que proporciona a clase son similares aos que xa vimos da clase `DataInputStream`. Ao igual que nesta clase teñen a seguinte forma:

Métodos xerais de lectura e escritura

```
readXxx(); //Métodos de lectura onde Xxx é o nome do tipo primitivo.
writeXxx(); //Métodos de escritura onde Xxx é o nome do tipo primitivo.
```

Ao igual que ocorría cos ficheiros binarios, todos os métodos de lectura lanzan a excepción `EOFException` cando por mor da lectura se alcanza o final do ficheiro.

Respecto ao funcionamento dos métodos de escritura, hai que aclarar, que cando se utiliza un método `write` polo medio do ficheiro, é dicir, cando a posición do punteiro non estea ao final do ficheiro, entón o método o que vai a facer é sobrescribir a información xa existente no arquivo, non a engade. Pola contra, se a posición do punteiro está ao final do arquivo, se utilizamos un método de escritura, entón si que escribirá ao final do arquivo e incrementará a lonxitude do mesmo.



Acceder ao enlace da API de Java para consultar todos os métodos proporcionados pola clase `RandomAccessFile`: <https://docs.oracle.com/javase/8/docs/api/>

Hai outros métodos que son máis específicos do uso desta clase que son os que se mostrar na seguinte táboa:

Método	Descrición
<code>long getFilePointer()</code>	Devolve a posición actual do punteiro medida desde o comezo do arquivo, a partir do cal se fará a seguinte lectura ou escritura.
<code>long length()</code>	Devolve a lonxitude do ficheiro.
<code>void seek(long posicion)</code>	Establece o desprazamento do punteiro, medido desde o comezo do arquivo, a partir do cal se fará a seguinte lectura ou escritura. O desprazamento pode situarse máis alá do final do arquivo, pero isto non fará que se modifique a lonxitude do arquivo. A lonxitude do arquivo só se modificará cando despois dunha escritura, o desprazamento se estableza máis alá do final do arquivo. Devolve unha excepción <code>IOException</code> se a posición indicada é menor que 0 ou se ocorre algún erro de entrada/saída.
<code>long setLength(long lonxitude)</code>	Establece a lonxitude do ficheiro. Pódense dar dúas situacións: - Se a lonxitude é maior que o tamaño dado polo método <code>length()</code>, entón se estenderá o arquivo. - Se a lonxitude é menor que o tamaño dado polo método <code>length()</code>, entón o ficheiro será truncado. Se a onde apunta actualmente o punteiro (valor dado polo método <code>getFilePointer()</code>) é maior que a nova lonxitude entón o modificarase o punteiro para que apunte á nova lonxitude.

Todos os métodos da táboa devolven unha excepción `IOException` se ocorre algún erro de entrada/saída.

A continuación temos un pequeno exemplo de lectura e escritura dun ficheiro de acceso aleatorio.

Exemplo de lectura e escritura dun ficheiro de acceso aleatorio

```
//Abrimos o ficheiro para lectura e escritura usando try-with-resources para
//que ao final do bloque se peche automaticamente o ficheiro.
try (RandomAccessFile ficheiro = new RandomAccessFile("ficheiro.dat", "rw")){
    //Comprobamos que o punteiro sinala ao comezo do ficheiro unha vez que se abre
    System.out.println("Posición actual do punteiro: "+ficheiro.getFilePointer());
    //Escribimo un valor de tipo enteiro
    ficheiro.writeInt(20);
    //Escribimos un valor de tipo float
    ficheiro.writeFloat(6.45f);
    //Comprobamos onde está colocado o punteiro despois das dúas escrituras
    System.out.println("Posición actual do punteiro: "+ficheiro.getFilePointer());
    //Comprobamos a lonxitude do ficheiro despois das dúas escrituras
    System.out.println("Tamaño do ficheiro: "+ficheiro.length());
    //Movemos o punteiro ao comezo do ficheiro para poder ler todo o seu contido
    ficheiro.seek(0);
    //Lemos os dous valores, primeiro o enteiro e logo o float
    System.out.println("Primeiro valor: "+ficheiro.readInt());
    System.out.println("Primeiro valor: "+ficheiro.readFloat());
} catch (FileNotFoundException fnfe) {
    System.out.println("ERRO: O arquivo indicado pode non ser un ficheiro.");
} catch (IOException ioe) {
    System.out.println("ERRO: Houbo un erro de entrada/saída.");
}
```

O resultado da execución é o seguinte:

```
run:
Posición actual do punteiro: 0
Posición actual do punteiro: 8
Tamaño do ficheiro: 8
Primeiro valor: 20
Primeiro valor: 6.45
BUILD SUCCESSFUL (total time: 0 seconds)
```



Realizar a Tarefa 5 “Xestión dun ficheiro de acceso aleatorio”, onde o alumnado deberá facer a xestión do alumnado dun centro educativo utilizando un ficheiro aleatorio. A xestión incluírá: a inserción de novos alumnos, a modificación dos seus datos e impresión de listados.

1.2.6 Serialización

A serialización consiste na transformación dun obxecto nunha secuencia de bytes que posteriormente poden ser lidos para reconstruír o obxecto orixinal. Unha vez que temos a secuencia de bytes, esta pode ser almacenada nun ficheiro.

O feito de gardar un obxecto e que siga existindo máis aló de cando se remate a execución da aplicación é o que se coñece como persistencia.

Un detalle importante para que un obxecto poda transformarse nunha secuencia de bytes é que este debe ser serializable. Para elo, ten que implementar a interface de Java `Serializable` que se atopa no paquete `java.io`. Esta interface está baleira e non contén ningún método.

A clase `Serializable` só serve para indicarlle á máquina virtual de Java que o obxecto que a implementa se pode serializar.

Todos os tipos básicos de Java, xunto cos arrays e os `Strings`, son serializables.

Unha das clases que vimos que derivan da clase `OutputStream` é a clase `ObjectOutputStream`. Esta clase é a que nos permite serializar un obxecto.

Como ocorría noutras clases, para crear un obxecto de tipo `ObjectOutputStream` necesitamos facelo desde un obxecto `FileOutputStream` asociado ao ficheiro onde vamos a almacenar a secuencia de bytes que se corresponde co obxecto.

O construtor dispoñible na clase é o seguinte:

Construtor

```
ObjectOutputStream(OutputStream nombre);
```

Este construtor lanza a excepción `IOException` que deberá ser tratada.

Unha vez que teñamos un obxecto de tipo `ObjectOutputStream` podemos invocar ao método de escritura de obxectos no ficheiro. Este método tamén lanza unha excepción `IOException`.

Método de escritura de obxectos

```
void writeObject(Object obxecto);
```

Vamos a ver un exemplo xenérico. Para elo consideremos que temos a seguinte clase:

Exemplo de clase serializable

```
package probafile;
import java.io.Serializable;
public class ClaseExemplo implements Serializable{
    //Atributos da clase
    //Métodos da clase
}
```

A continuación, móstrase o código que crea dous obxectos da clase anterior e os almacena en forma de secuencia de bytes nun ficheiro.

Exemplo de escritura nun ficheiro de obxectos

```
public class ProbaFile {
    public static void main(String[] args) {
        ObjectOutputStream saida = null;
        ClaseExemplo ce;
        try {
            //Créase o ficheiro e o obxecto para serializar
            saida = new ObjectOutputStream(new FileOutputStream("datos.dat"));
            //Créase o primeiro obxecto
            ce = new ClaseExemplo();
            //Escribese o obxecto no ficheiro
            saida.writeObject(ce);
            //Créase o segundo obxecto
            ce = new ClaseExemplo();
            //Se escribe el objeto en el fichero
            saida.writeObject(ce);
        } catch (FileNotFoundException e) {
            System.out.println("ERRO: Problemas co ficheiro");
        } catch (IOException e) {
            System.out.println("ERRO: Problemas de entrada/saída");
        } finally {
            try {
                if(saida!=null) {
                    saida.close();
                }
            } catch (IOException e) {
                System.out.println("ERRO: Problemas ao pechar o ficheiro");
            }
        }
    }
}
```



Realizar a Tarefa 6 “Serialización de obxectos”, onde o alumnado deberá crear obxectos a partir da información de entrada proporcionada polo usuario e almacenalos nun ficheiro.

Unha vez que vimos a escritura de obxectos en ficheiros, necesitamos saber como ler os obxectos almacenados nun ficheiro como unha secuencia de bytes. Para elo dispomos da clase `ObjectInputStream` derivada de `InputStream`.

Para poder realizar a lectura é preciso crear un obxecto da clase `ObjectInputStream`. O construtor que nos permite crear o obxecto necesita facer referencia ao ficheiro desde o que realizar a lectura, por iso hai que pasarlle un obxecto `FileInputStream` asociado ao ficheiro que se quere ler.

Construtor

```
ObjectInputStream(InputStream nombre);
```

Ao igual que na creación do obxecto para a escritura, a creación dun obxecto para a lectura tamén lanza a excepción `IOException`.

A clase dispón dun método específico para a lectura de obxectos desde un ficheiro. Este método devolve un obxecto do ficheiro de tipo `Object`, así que é necesario facer un casting sobre el para gardalo nunha variable do tipo adecuado e así poder tratala na nosa aplicación específica. Este método lanza varias excepcións das que hai que tratar polo menos as seguintes:

- `IOException`.
- `ClassNotFoundException`: lanza esta excepción cando non atopa a clase do obxecto serializado.

Partindo do exemplo anterior, onde escribíamos dous obxectos nun ficheiro, vamos a ver como se realizaría de forma xenérica a lectura para a súa impresión por pantalla.

Exemplo de lectura dun ficheiro de obxectos

```
public class ProbaFile {
    public static void main(String[] args) {
        ObjectInputStream entrada = null;
        ClaseExemplo ce;
        try {
            //Créase o fluxo de entrada para ler obxectos serializados
            entrada = new ObjectInputStream(new FileInputStream("datos.dat"));
            //Lese o primeiro obxecto contido no ficheiro
            ce = (ClaseExemplo) entrada.readObject();
            System.out.println(ce.toString());
            //Lese o segundo obxecto contido no ficheiro
            ce = (ClaseExemplo) entrada.readObject();
            System.out.println(ce.toString());
        } catch (ClassNotFoundException e) {
            System.out.println("ERRO: Non se pode atopar a clase da serialización");
        } catch (IOException e) {
            System.out.println("ERRO: Problemas de entrada/saída");
        } finally {
            try {
                if(entrada!=null) {
                    entrada.close();
                }
            } catch (IOException e) {
                System.out.println("ERRO: Problemas ao pechar o ficheiro");
            }
        }
    }
}
```



Realizar a Tarefa 7 “Lectura e modificación de obxectos almacenados nun ficheiro”, onde o alumnado deberá ler os obxectos almacenados nun ficheiro, cargalos en memoria e manipularlos.

1.3 Tarefas

As tarefas propostas para esta actividade son as seguintes:

- **Tarefa 1. Información dun directorio.** Nesta tarefa o alumnado deberá crear unha aplicación na que dado un directorio se podan realizar distintas operacións sobre o mesmo.
- **Tarefa 2. Listado de empregados almacenados nun ficheiro ordenados.** Nesta tarefa o alumnado deberá ler un ficheiro no que se almacenan os datos dos empregados dunha empresa e mostrar a información ordenada.
- **Tarefa 3. Xestión dun ficheiro de empregados.** Nesta tarefa o alumnado deberá modificar a tarefa 2, para realizar a lectura dun ficheiro e para a súa escritura con información nova.
- **Tarefa 4. Copia dun ficheiro binario.** Nesta tarefa o alumnado deberá crear un ficheiro binario cos datos dados polo usuario e crear unha copia deste ficheiro, así como mostrar os datos por pantalla.
- **Tarefa 5. Xestión dun ficheiro de acceso aleatorio.** Nesta tarefa o alumnado deberá utilizar un ficheiro aleatorio para xestionar os datos do alumnado dun centro educativo, de maneira que se podan inserir novos alumnos, modificar os seus datos e mostrar listados.

- Tarefa 6. **Serialización de obxectos**. Nesta tarefa o alumnado deberá almacenar nun ficheiro unha serie de obxectos creados a partir da información dada polo usuario.
- Tarefa 7. **Lectura e modificación de obxectos almacenados nun ficheiro**. Nesta tarefa o alumnado deberá ler a información contida nun ficheiro que almacena obxectos, cargala en memoria e logo traballar con ela.

1.3.1 Tarefa 1. Información dun directorio

Enunciado

Crear unha aplicación que permita realizar certas operacións sobre un directorio dado. O directorio de partida solicitaráselle ao usuario. Sobre este directorio poderán realizarse as seguintes operacións:

- Listar contido: deberá listar o contido do directorio. Por un lado listaranse os arquivos, para elo indicarse o número total de arquivos, o nome e tamaño de cada un. Por outro lado listaranse os directorios, de maneira que se indique o número total de directorios e o nome de cada un deles.
- Crear un directorio novo: solicitaráselle ao usuario que introduza o nome do novo directorio e crease dentro do directorio actual o novo directorio.
- Borrar o directorio: borrarase o directorio pero soamente cando estea baleiro. Antes de borrarlo pedirase ao usuario unha confirmación.
- Renomear o directorio: pedirase ao usuario o novo nome do directorio e farase o renomeado.
- Introducir unha nova ruta a un directorio: solicitará de novo ao usuario unha ruta para traballar con ela.
- Saír da aplicación: só cando se seleccione esta opción se sairá do programa.

Deberase ter en conta que para realizar calquera das operacións anteriores a cadea introducida polo usuario deberá ser un directorio e non un ficheiro e este debe existir.

Sempre que sexa oportuno, deberase comprobar se a operación tivo éxito. En calquera caso, darase información ao usuario.

Solución

A continuación móstrase o código da aplicación:

```
package ud05_a02_tarefal;
import java.io.File;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.Scanner;
public class UD05_A02_Tarefal {
    public static void main(String[] args) {
        String ruta;
        File directorio;
        byte opcion = 0;
        boolean sair = false;
        ArrayList<String> dirs = new ArrayList<>();
        ArrayList<File> arqs = new ArrayList<>();
        do {
            ruta = lerString("Introduce a ruta ao directorio: ");
            directorio = new File(ruta);
            do {
                if (directorio.isDirectory()) {
                    System.out.println("MENÚ");
```

```

System.out.println("\t1. Listar contido.");
System.out.println("\t2. Crear directorio.");
System.out.println("\t3. Borrar directorio.");
System.out.println("\t4. Renomear directorio.");
System.out.println("\t5. Introducir nova ruta.");
System.out.println("\t6. Saír.");
opcion = validarOpcion(1, 6);
switch (opcion) {
    case 1: //Listar contido
        File[] listado = directorio.listFiles();
        File elemento;
        dirs.clear();
        arqs.clear();
        for (File listado1 : listado) {
            if (listado1.isDirectory()) {
                dirs.add(listado1.getName());
            } else {
                arqs.add(listado1);
            }
        }
        System.out.println("\tArquivos do directorio: "+arqs.size());
        Iterator it = arqs.iterator();

        for ( ; it.hasNext(); ){
            elemento = (File) it.next();
            System.out.print("\t\t" + (elemento.getName()));
            System.out.print("\t\t" + (elemento.length()) + " B\n");
        }
        System.out.println("\tDirectorios do directorio: "+dirs.size());

        it = dirs.iterator();
        for ( ; it.hasNext(); ){
            System.out.println("\t\t"+((String) it.next()));
        }
        break;
    case 2: //Crear directorio
        ruta = lerString("\tIntroduce o nome do novo directorio: ");
        ruta = directorio.getAbsolutePath() + File.separatorChar + ruta;

        System.out.println("Ruta: "+ruta);
        File novo = new File(ruta);
        if (!novo.exists()) {
            if (novo.mkdir()) {
                System.out.println("\tO novo directorio foi "
                    + "creado correctamente");
            } else {
                System.out.println("\tERRO: Non se puido crear "
                    + "o novo directorio.");
            }
        } else {
            System.out.println("\tNon se creou a directorio. "
                + "Xa existe.");
        }
        break;
    case 3: //Borrar directorio
        byte confirmacion;
        System.out.println("\tEstá seguro de querer eliminar "
            + "o directorio? [0-Non ; 1-Si] ");
        confirmacion = validarOpcion(0, 1);
        if (confirmacion == 1) {
            boolean rdo = directorio.delete();
            if (rdo) {
                System.out.println("\tO directorio foi borrado.");
                //Pomos a opcion a 5, porque como o directorio xa non
                //existe, así se solicita que se introduza unha nova
                //ruta
                opcion = 5;
            } else {
                System.out.println("\tERRO: non se puido borrar "
                    + "o directorio, non está baleiro.");
            }
        }
        break;
    case 4: //Renomear directorio
        boolean rdo;
        ruta = lerString("\tIntroduce o novo nome do directorio: ");
        ruta = directorio.getParent() + File.separatorChar + ruta;
        rdo = directorio.renameTo(new File(ruta));
        if (rdo) {
            System.out.println("\tO nome do directorio foi cambiado "

```

```

        + "correctamente");
        //Asignamos a nova ruta a directorio, con renameTo
        //non se cambia e segue tendo a antiga.
        directorio = new File(ruta);
    } else {
        System.out.println("\tAVISO: houbo algún erro, non se "
            + "puido cambiar o nome do directorio");
    }
    break;
case 5://Introducir nova ruta
    break;
case 6:
    sair = true;
    break;
    }
    } else {
        System.out.println("\tERRO: a ruta non é correcta ou non é un "
            + "directorio.");
    }
    } while ((opcion != 5) && (opcion != 6));
}while (!sair);
}

public static byte validarOpcion(int min, int max) {
    byte opcion = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);
    do {
        repetir = false;
        System.out.print ("Seleccione unha opción: ");
        if (sc.hasNextByte()) {
            opcion = sc.nextByte();
            if (opcion < min || opcion > max) {
                System.out.println("\tATENCIÓN! Opción non dispoñible. "
                    + "Volva a seleccionar. ");
                repetir = true;
            }
        } else {
            System.out.println("\tERRO: Débese introducir unha entre "
                + min + " e " + max);
            sc.next();
            repetir = true;
        }
    } while (repetir);
    return opcion;
}

public static String lerString(String cadea) {
    String valor = "";
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextLine()) {
            valor = sc.nextLine();
            repetir = (valor.isEmpty());
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir algún texto. "
                + "\n\t\tVolva a introducir o valor: ");
        }
    }
    return valor;
}
}
}

```

Nas seguintes capturas pode verse parte do resultado da execución.

Nesta primeira imaxe pode comprobarse como se introduce o nome do directorio e como se lista o seu contido.

```

run:
Introduce a ruta ao directorio: C:\Users\Public\Documents\Directorio
MENÚ
    1. Listar contido.
    2. Crear directorio.
    3. Borrar directorio.
    4. Renomear directorio.
    5. Introducir nova ruta.
    6. Saír.
Seleccione unha opción: 1
    Archivos do directorio: 5
        debuxo1.odg          8368 B
        folla1.ods           6557 B
        presentacion1.odp    8893 B
        texto1.odt           7405 B
        texto2.odt           7405 B
    Directorios do directorio: 4
        dir1
        dir2
        dir3
        dir4
MENÚ
    1. Listar contido.

```

Nesta outra imaxe vese como crear un novo directorio.

```

MENÚ
    1. Listar contido.
    2. Crear directorio.
    3. Borrar directorio.
    4. Renomear directorio.
    5. Introducir nova ruta.
    6. Saír.
Seleccione unha opción: 2
    Introduce o nome do novo directorio: novoDir
    Ruta: C:\Users\Public\Documents\Directorio\novoDir
    O novo directorio foi creado correctamente
MENÚ
    1. Listar contido

```

Para ver que o directorio se creou correctamente volvemos a listar o contido do directorio actual como se ve na seguinte imaxe:

```

MENÚ
    1. Listar contido.
    2. Crear directorio.
    3. Borrar directorio.
    4. Renomear directorio.
    5. Introducir nova ruta.
    6. Saír.
Seleccione unha opción: 1
    Archivos do directorio: 5
        debuxo1.odg          8368 B
        folla1.ods           6557 B
        presentacion1.odp    8893 B
        texto1.odt           7405 B
        texto2.odt           7405 B
    Directorios do directorio: 5
        dir1
        dir2
        dir3
        dir4
        novoDir

```


1.3.2 Tarefa 2. Listado de empregados almacenados nun ficheiro ordenados

Enunciado

Deberase ler o contido do ficheiro que contén os datos dos empregados dunha empresa. Almacenaranse todos os empregados en memoria e listaranse por orde alfabética e tamén por departamento, e dentro de cada departamento por orde alfabética.

Deberase valorar a forma máis adecuada de almacenar a todos os empregados en memoria.

A estrutura do ficheiro será como a do ficheiro de exemplo do apartado 1.2.1. empregados.dat

Solución

Código da clase Empregado:

```
package ud05_a02_tarefa2;
public class Empregado {
    String nome;
    String dni;
    int idade;
    String departamento;

    Empregado(String n, String d, int i, String dep) {
        nome = n;
        dni = d;
        idade = i;
        departamento = dep;
    }

    @Override
    public String toString() {
        return "\t"+nome+"\t"+dni+"\t"+idade+"\t"+departamento;
    }
}
```

Código da clase de ordenación alfabética OrdeAlfabetico:

```
package ud05_a02_tarefa2;
import java.util.Comparator;
public class OrdeAlfabetico implements Comparator <Empregado>{
    @Override
    public int compare(Empregado e1, Empregado e2) {
        return e1.nome.compareToIgnoreCase(e2.nome);
    }
}
```

Código da clase de ordenación por departamento e por orde alfabética OrdeDepartamento:

```
package ud05_a02_tarefa2;
import java.util.Comparator;
public class OrdeDepartamento implements Comparator <Empregado>{
    @Override
    public int compare(Empregado e1, Empregado e2) {
        int valor;
        if (e1.departamento.equalsIgnoreCase(e2.departamento)) {
            valor = e1.nome.compareToIgnoreCase(e2.nome);
        } else {
            valor = e1.departamento.compareToIgnoreCase(e2.departamento);
        }
        return valor;
    }
}
```

Código da clase principal:

```
package ud05_a02_tarefa2;
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
```

```

import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
public class UD05_A02_Tarefa2 {
    public static void main(String[] args) {
        String linha = "";
        ArrayList<Empregado> empregados = new ArrayList<>();
        Iterator it;
        try (BufferedReader buffer = new BufferedReader(new FileReader("empregados.txt")))
        {
            linha = buffer.readLine();
            while (linha != null) {
                engadirLinha(empregados, linha);
                linha = buffer.readLine();
            }
            System.out.println("Empregados ordenados por orde alfabética:");
            empregados.sort(new OrdeAlfabetico());
            it = empregados.iterator();
            while (it.hasNext()) {
                Empregado emp = (Empregado)it.next();
                System.out.println(emp);
            }
            System.out.println("Empregados ordenados por departamento e por orde "
                + "alfabética :");
            empregados.sort(new OrdeDepartamento());
            it = empregados.iterator();
            while (it.hasNext()) {
                Empregado emp = (Empregado)it.next();
                System.out.println(emp);
            }
            System.out.println("");
        } catch (FileNotFoundException e) {
            System.out.println("ERRO: O ficheiro é un directorio ou ben non existe.");
        } catch (IOException e) {
            System.out.println("ERRO: Ocorreu un erro na lectura do ficheiro");
        }
    }

    public static void engadirLinha(ArrayList<Empregado> lista, String lin) {
        String[] valores = lin.split(";");
        lista.add(new Empregado(valores[0],valores[1],Integer.valueOf(valores[2]),
            valores[3]));
    }
}

```

O resultado da execución do programa, tendo en conta que o ficheiro empregados.txt contén os mesmos datos que o que se mostra na imaxe do apartado 2.2.1, é o seguinte:

```

run:
Empregados ordenados por orde alfabética:
Ana      222222222B      32      Programación
Francisco 555555555E      33      Recepción
Marta    666666666F      35      Análise&Deseño
Pedro    111111111A      25      RR.HH
Sofía    777777777G      47      RR.HH.
Xan       333333333C      50      Análise&Deseño
Eduardo  444444444D      29      Administración
Empregados ordenados por departamento e por orde alfabética :
Eduardo  444444444D      29      Administración
Marta    666666666F      35      Análise&Deseño
Xan       333333333C      50      Análise&Deseño
Ana      222222222B      32      Programación
Francisco 555555555E      33      Recepción
Pedro    111111111A      25      RR.HH
Sofía    777777777G      47      RR.HH.

BUILD SUCCESSFUL (total time: 0 seconds)

```

1.3.3 Tarefa 3. Xestión dun ficheiro de empregados

Enunciado

Deberase retomar a tarefa 2 para completala tal e como se indica a continuación.

Deberase modificar a aplicación creada na tarefa 2 para que permita realizar as seguintes operacións, segundo a selección do usuario:

- Listar os empregados ordenados alfabeticamente.
- Listar os empregados ordenados por departamento, e dentro deste por orde alfabético.
- Engadir un novo empregado ao listado.
- Borrar un empregado.
- Saír da aplicación.

Para engadir un novo empregado deberase solicitar ao usuario o nome, dni, idade e departamento e teranse en conta as seguintes condicións:

- O DNI debe ser correcto.
- O departamento deberá ser un entre os seguintes: RR.HH., Administración, Análise&Deseño ou Programación.

Para borrar un empregado preguntárase polo dni do empregado que se desexa borrar, e se este existe no listado de empregados será borrado.

Saír da aplicación implica tanto o peche do programa como o volcado da información dos empregados que se atopa en memoria ao mesmo ficheiro desde o que se fixo a lectura.

Solución

Partimos do ficheiro de texto do exercicio anterior.

As clases `Empleado`, `OrdeAlfabetico` e `OrdeDepartamento` son as mesmas que na tarefa anterior.

A clase que cambia é a principal. Móstrase a continuación o código:

```
package ud05_a02_tarefa3;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.Scanner;
public class UD05_A03_Tarefa3 {
    public static void main(String[] args) {
        byte opcion;
        boolean sair = false;
        ArrayList<Empleado> empregados = new ArrayList<>();
        lerFicheiro(empregados);
        do {
            System.out.println("MENÚ");
            System.out.println("\t1. Mostrar empregados (alfabeticamente).");
            System.out.println("\t2. Mostrar empregados (por departamento).");
            System.out.println("\t3. Engadir empregado.");
            System.out.println("\t4. Borrar empregado.");
            System.out.println("\t5. Sair.");
            opcion = validarOpcion(1, 5);
            switch (opcion) {
                case 1: //Mostrar empregados (alfabeticamente)
```

```

        mostrarEmpOrdenadosAlf(emplegados);
        break;
    case 2: //Mostrar empregados (por departamento)
        mostrarEmpOrdenadosDep(emplegados);
        break;
    case 3: //Engadir empregado
        String nome, dni, departamento;
        int idade;
        System.out.println("\t== NOVO EMPREGADO ==");
        nome = lerString("\tNome: ");
        dni = lerString("\tDNI: ");
        while (!comprobarDNI(dni)) {
            System.out.println("\tERRO: o dni é incorrecto."
                + "Volva a introducilo.");
            dni = lerString("\tDNI: ");
        }
        idade = lerInt("\tIdade: ");
        System.out.println("\tDepartamento: (Elixa un dos seguintes)");
        System.out.println("\t\t1. RR.HH.");
        System.out.println("\t\t2. Administración.");
        System.out.println("\t\t3. Análise e deseño.");
        System.out.println("\t\t4. Programación.");
        System.out.print("\t\t\tOpción elixida: ");
        opcion = validarOpcion(1, 4);
        departamento = "";
        switch (opcion) {
            case 1:
                departamento = "RR.HH.";
                break;
            case 2:
                departamento = "Administración";
                break;
            case 3:
                departamento = "Análise&deseño";
                break;
            case 4:
                departamento = "Programación";
                break;
        }
        empregados.add(new Empleado(nome, dni, idade, departamento));
        break;
    case 4: //Borrar empregado
        boolean borrado = false;
        if (emplegados.isEmpty()) {
            System.out.println("Operación non válida. Non hai empregados");
        } else {
            Iterator it = empregados.iterator();
            System.out.println("\t== BORRAR EMPREGADO ==");
            dni = lerString("Introduza o DNI do empregado a borrar: ");
            for (; it.hasNext(); ) {
                Empleado emp = (Empleado) it.next();
                if (emp.dni.equalsIgnoreCase(dni)) {
                    it.remove();
                    System.out.println("\tBorrarse o empregado con DNI: "
                        + emp.dni);
                    borrado = true;
                }
            }
            if (!borrado) {
                System.out.println("\tNon se atopou o empregado.");
            }
        }
        break;
    case 5:
        volcarFicheiro(emplegados);
        sair = true;
        break;
    }
} while (!sair);
}

public static void lerFicheiro(ArrayList<Empleado> lista) {
    String linha = "";
    try (BufferedReader buffer = new BufferedReader(new FileReader("emplegados.txt")))
    {
        linha = buffer.readLine();
        while (linha != null) {
            engadirLinha(lista, linha);
            linha = buffer.readLine();
        }
    }
}

```

```

    } catch (FileNotFoundException e) {
        System.out.println("ERRO: O ficheiro é un directorio ou ben non existe.");
    } catch (IOException e) {
        System.out.println("ERRO: Ocorreu un erro na lectura do ficheiro");
    }
}

public static void volcarFicheiro(ArrayList<Empleado> lista) {
    //Como volcamos completamente o contido do arrayList, cando volcamos
    //reescribimos todo o ficheiro
    try (BufferedWriter buffer = new BufferedWriter(
        new FileWriter("empleados.txt", false))) {
        Iterator it;
        it = lista.iterator();
        while (it.hasNext()) {
            Empleado emp = (Empleado)it.next();
            buffer.write(emp.nome + ";" + emp.dni + ";" + emp.idade + ";"
                + emp.departamento);
            buffer.newLine();
        }
    } catch (IOException e) {
        System.out.println("ERRO: Ocorreu un erro na lectura do ficheiro");
    }
}

public static void engadirLinha(ArrayList<Empleado> lista, String lin) {
    String[] valores = lin.split(";");
    lista.add(new Empleado(valores[0],valores[1],Integer.valueOf(valores[2]),
        valores[3]));
}

public static void mostrarEmpOrdenadosAlf(ArrayList<Empleado> lista) {
    Iterator it;
    System.out.println("\t== LISTADO EMPREGADOS (alfabético)==");
    lista.sort(new OrdeAlfabetico());
    it = lista.iterator();
    while (it.hasNext()) {
        Empleado emp = (Empleado)it.next();
        System.out.println(emp);
    }
}

public static void mostrarEmpOrdenadosDep(ArrayList<Empleado> lista) {
    Iterator it;
    System.out.println("\t== LISTADO EMPREGADOS (por departamento)==");
    lista.sort(new OrdeDepartamento());
    it = lista.iterator();
    while (it.hasNext()) {
        Empleado emp = (Empleado)it.next();
        System.out.println(emp);
    }
    System.out.println("");
}

public static byte validarOpcion(int min, int max) {
    byte opcion = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);
    do {
        repetir = false;
        System.out.print ("Seleccione unha opción: ");
        if (sc.hasNextByte()) {
            opcion = sc.nextByte();
            if (opcion < min || opcion > max) {
                System.out.println("\tATENCIÓN! Opción non dispoñible. "
                    + "Volva a seleccionar. ");
                repetir = true;
            }
        } else {
            System.out.println("\tERRO: Débese introducir unha entre "
                + min + " e " + max);
            sc.next();
            repetir = true;
        }
    } while (repetir);
    return opcion;
}

```

```

public static String lerString(String cadea) {
    String valor = "";
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextLine()) {
            valor = sc.nextLine();
            repetir = (valor.isEmpty());
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir algún texto. "
                + "\n\t\tVolva a introducir o valor: ");
        }
    }
    return valor;
}

public static int lerInt(String cadea) {
    int valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextInt()) {
            valor = sc.nextInt();
            repetir = (valor < 1);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor numérico. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
    return valor;
}

public static boolean comprobarDNI (String num) {
    boolean correcto = false;
    if (num.length() == 9){
        String letra = num.substring(8);
        num = num.substring(0,8);
        if ((num.matches("\\d{8}")
            && (letra.matches("[a-zA-Z]{1}")))) {
            int i = Integer.parseInt(num);
            char a = letra.charAt(0);
            if (validarDNI(i,a)) {
                correcto = true;
            }
        }
    }
    return correcto;
}

public static boolean validarDNI(int num, char letra) {
    char letras[] = {'T','R','W','A','G','M','Y','F','P','D','X','B','N','J',
        'Z','S','Q','V','H','L','C','K','E'};
    int resto = num % 23;
    return letras[resto] == Character.toUpperCase(letra);
}
}

```

Na seguinte imaxe vese o menú e como é o resultado da execución de engadir un novo empregado:

```

run:
MENÚ
    1. Mostrar empregados (alfabeticamente).
    2. Mostrar empregados (por departamento).
    3. Engadir empregado.
    4. Borrar empregado.
    5. Saír.
Seleccione unha opción: 3
    == NOVO EMPREGADO ==
    Nome: Raquel
    DNI: 45678984H
    Idade: 50
    Departamento: (Elixa un dos seguintes)
        1. RR.HH.
        2. Administración.
        3. Análise e deseño.
        4. Programación.
    Opción elixida: Seleccione unha opción: 1
MENÚ
    1. Mostrar empregados (alfabeticamente).

```

1.3.4 Tarefa 4. Copia dun ficheiro binario

Enunciado

Deberase crear unha aplicación que realice as seguintes tres operacións:

- Crear un ficheiro binario con datos de estudantes: os datos dos estudantes solicitaranse ao usuario pola consola e iranse escribindo nun ficheiro, cuxo nome sexa dado previamente polo usuario. Os datos que se solicitarán de cada estudante son os seguintes:

– Nome.

– Idade.

– Nota media: será un valor de tipo `float`, e representa a media de todos os cursos anteriores ao curso actual.

– Booleano indicando se o alumno repite ou non curso.

Rematarase de ler os datos cando como nome se introduza o texto “FIN”.

- Facer unha copia do ficheiro: unha vez rematada a lectura completa dos datos no ficheiro anterior, farase unha copia completa do ficheiro noutro ficheiro, cuxo nome se lle solicitará previamente ao usuario.
- Mostrar os datos do ficheiro por pantalla.

Solución

O código da aplicación podería ser o que se mostra a continuación. Hai que indicar que non está completo. Non se inclúen comprobacións para non facer complicada a lectura, pero deberíanse establecer controis sobre a lectura dos datos de entrada. Por exemplo, se non se introduce un enteiro cando se supón que se vai ler un valor de tipo enteiro o programa deixa de funcionar de forma abrupta.

```

package ud05_a02_tarefa4;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.EOFException;
import java.io.FileInputStream;
import java.io.FileNotFoundException;

```

```

import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Scanner;
public class UD05_A02_Tarefa4 {
    static Scanner sc = new Scanner(System.in);
    static String texto = "";
    public static void main(String[] args) {
        boolean criado;
        System.out.println(" == CRIAR FICHEIRO ESTUDIANTES ==");
        criado = criarFicheiro();

        System.out.println(" == COPIAR FICHEIRO ESTUDIANTES ==");
        if (criado) {
            criado = copiarFicheiro();
        } else {
            System.out.println("\tERRO: Non se pode crear a copia do ficheiro. "
                + "Houbo un erro na creaci3n.");
        }
    }

    public static boolean criarFicheiro() {
        String nome;
        int idade;
        float nota;
        byte repite;
        boolean continuar = true;
        boolean resultado = false;
        System.out.print("Introduza o nome do ficheiro de alumnos: ");
        texto = sc.nextLine();
        try (DataOutputStream dos = new DataOutputStream(new FileOutputStream(texto))) {
            while (continuar) {
                System.out.println("\t == NOVO ESTUDIANTE ==");
                System.out.print("\t\t Nome [FIN para saír]: ");
                nome = sc.nextLine();
                if (!nome.equalsIgnoreCase("FIN")) {
                    dos.writeUTF(nome);
                    System.out.print("\t\t Idade: ");
                    idade = sc.nextInt();
                    sc.nextLine();
                    dos.writeInt(idade);
                    System.out.print("\t\t Nota: ");
                    nota = sc.nextFloat();
                    sc.nextLine();
                    dos.writeFloat(nota);
                    System.out.print("\t\t Repite [0-Non 1-Si]: ");
                    repite = sc.nextByte();
                    sc.nextLine();
                    if (repite == 0) {
                        dos.writeBoolean(false);
                    } else {
                        dos.writeBoolean(true);
                    }
                } else {
                    continuar = false;
                }
            }
            resultado = true;
        } catch (FileNotFoundException e) {
            System.out.println("\tERRO: O arquivo indicado non é un ficheiro.");
            resultado = false;
        } catch (IOException e) {
            System.out.println("\tERRO: Houbo en erro de entrada/saída.");
            resultado = false;
        }
        return resultado;
    }

    public static boolean copiarFicheiro() {
        String nome;
        int idade;
        float nota;
        boolean repite;
        boolean resultado = false;
        String copia;
        System.out.print("Introduza o nome da copia do ficheiro de alumnos: ");
        copia = sc.nextLine();
        DataInputStream entrada = null;
        DataOutputStream saida = null;
        try {
            entrada = new DataInputStream(new FileInputStream(texto));

```



```

saida = new DataOutputStream(new FileOutputStream(copia));
System.out.println("\tNOME\tIDADE\tNOTA\tREPITE");
while (true) {
    nome = entrada.readUTF();
    System.out.print ("\t"+nome);
    saida.writeUTF(nome);

    idade = entrada.readInt();
    saida.writeInt(idade);
    System.out.print ("\t"+idade);

    nota = entrada.readFloat();
    saida.writeFloat(nota);
    System.out.print ("\t"+nota);

    repite = entrada.readBoolean();
    saida.writeBoolean(repite);
    System.out.print ("\t"+repite+"\n");
}
} catch (FileNotFoundException fnfex) {
    System.out.println("\tERRO: O ficheiro é un directorio.");
    resultado = false;
} catch (EOFException eofex) {
    System.out.println("\tFin da copia.");
    resultado = true;
} catch (IOException ioex) {
    System.out.println("\tERRO: problemas na entrada/saída.");
    resultado = false;
}finally {
    try {
        if(entrada != null) {
            entrada.close();
        }
        if (saida != null) {
            saida.close();
        }
    } catch (IOException ex) {
        System.out.println("\tERRO: Houbo en erro de entrada/saída ao "
            + "pechar o ficheiro.");
    }
}

return resultado;
}
}

```

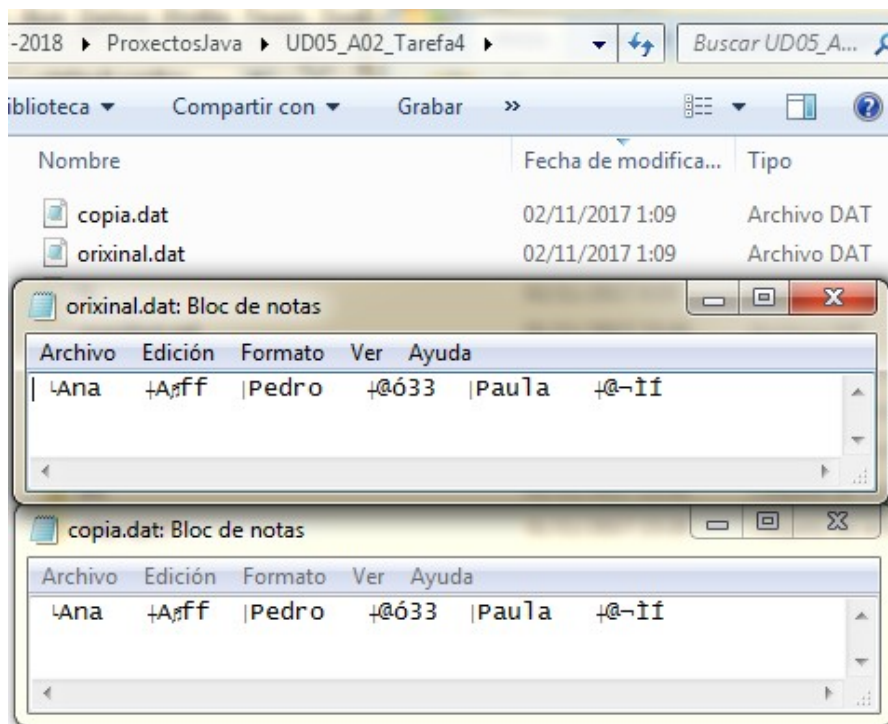
O resultado da execución é o seguinte:

```

run:
== CREAM FICHEIRO ESTUDIANTES ==
Introduza o nome do ficheiro de alumnos: orixinal.dat
== NOVO ESTUDIANTE ==
Nome [FIN para saír]: Ana
Idade: 16
Nota: 8,9
Repite [0-Non 1-Si]: 0
== NOVO ESTUDIANTE ==
Nome [FIN para saír]: Pedro
Idade: 16
Nota: 7,6
Repite [0-Non 1-Si]: 0
== NOVO ESTUDIANTE ==
Nome [FIN para saír]: Paula
Idade: 16
Nota: 5,4
Repite [0-Non 1-Si]: 1
== NOVO ESTUDIANTE ==
Nome [FIN para saír]: fin
== COPIAR FICHEIRO ESTUDIANTES ==
Introduza o nome da copia do ficheiro de alumnos: copia.dat
NOME    IDADE    NOTA    REPITE
Ana      16        8.9     false
Pedro    16        7.6     false
Paula    16        5.4     true
Fin da copia.
BUILD SUCCESSFUL (total time: 50 seconds)

```

No sistema de ficheiros como resultado teremos dous ficheiros chamados orixinal.dat e copia.dat con contidos idénticos.



1.3.5 Tarefa 5. Xestión dun ficheiro de acceso aleatorio

Enunciado

Tomando como referencia o formato do ficheiro da tarefa 4, deberase crear unha aplicación que xestione un ficheiro de alumnos. O acceso a este ficheiro deberá ser aleatorio en lugar de secuencial.

A aplicación mostrará un menú que permitirá realizar as seguintes operacións:

- Dar de alta un novo alumno: de forma similar ao que se fixo na tarefa 4 e engadindo o campo DNI, que se deberá comprobar que é correcto.
- Modificar os datos dun alumno existente no ficheiro: para elo solicitarase ao usuario que introduza o dni do alumno a procurar (xa que non pode haber dous alumnos que teñan o mesmo dni). Se se atopa o alumno mostraranse os datos actuais e solicitaráselle ao usuario que indique o campo a modificar, que poderá ser calquera: nome, dni, idade, nota e se repite ou non. En base ao campo a modificar, solicitarase o valor que corresponda e farase a modificación no ficheiro.
- Mostrar un listado co alumnado que é repetidor: deberá seleccionarse do ficheiro todos aqueles alumnos que estean repetidos e mostraranse os seus datos.
- Mostrar un listado completo: neste caso, mostraranse todos os datos de todos os alumnos almacenados no ficheiro.
- Saír: que permitirá ao usuario saír da aplicación.

Solución

A continuación móstrase o código que daría unha posible solución ao enunciado da tarefa. Podería mellorarse o código creando métodos que fagan funcións concretas, para que o fluxo do programa principal non fose complicado de ler. Optouse por esta opción para que a lectura sexa continua e facilite a comprensión das operacións realizadas.

```
package ud05_a02_tarefa5;
import java.io.EOFException;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.RandomAccessFile;
import java.util.Scanner;
public class UD05_A02_Tarefa5 {
    static Scanner sc = new Scanner(System.in);
    public static final int TAM_LINHA = 32;
    public static void main(String[] args) {
        byte opcion = 0;
        boolean sair = false;
        String nome;
        String dni;
        int idade;
        float nota;
        byte repite;
        boolean repetidor;
        try (RandomAccessFile ficheiro = new RandomAccessFile("ficheiro.dat", "rw")){

            do {
                System.out.println("MENÚ");
                System.out.println("\t1. Engadir alumno.");
                System.out.println("\t2. Modificar datos alumno.");
                System.out.println("\t3. Listado alumnado repetidor.");
                System.out.println("\t4. Listado completo.");
                System.out.println("\t5. Saír.");
                opcion = validarOpcion(1, 5);
                switch (opcion) {
                    case 1: //Engadir alumno
                        System.out.println("\t == NOVO ESTUDIANTE ==");
                        nome = lerString("\t Nome (Máximo 10 caracteres): ")
```

```

        + "          ";
ficheiro.writeUTF(nome.substring(0, 10));
dni = lerString("\t   Dni: ");
while (!comprobarDNI(dni)) {
    System.out.println("\tERRO: o dni é incorrecto."
        + "Volva a introducilo.");
    dni = lerString("\tDNI: ");
}
ficheiro.writeUTF(dni);
idade = lerInt("\t   Idade: ");
ficheiro.writeInt(idade);
nota = lerFloat("\t   Nota: ");
ficheiro.writeFloat(nota);
repite = lerByte("\t   Repite [0-Non 1-Si]: ");
while (repite != 0 && repite != 1) {
    System.out.println("\tERRO: só opción 0 ou 1."
        + "Volva a introducir [0-Non 1-Si]:");
    repite = lerByte("\t   Repite [0-Non 1-Si]: ");
}
if (repite == 0) {
    ficheiro.writeBoolean(false);
} else {
    ficheiro.writeBoolean(true);
}
break;
case 2: //Modificar datos alumno
boolean atopado = false;
int pos = 0; //Almacena a liña actual de lectura
String procura = lerString("\t   Introduce o DNI do alumno"
    + " a procurar: ");
while (!comprobarDNI(procura)) {
    System.out.println("\tERRO: o dni é incorrecto."
        + "Volva a introducilo.");
    procura = lerString("\tDNI: ");
}
try {
    //Colócase o punteiro para a primeira lectura do dni
    ficheiro.seek(12);
    System.out.println("\t == RESULTADO ==");
    System.out.printf("\t%s\t%s\t%s\t%s\t%s\n", "   NOME   "
        , "   DNI   ", "IDADE", "NOTA", "REPITE");
    //Lee o ficheiro ata que chega ao final ou xa atopou
    //a un alumno co dni indicados
    while (true && !atopado) {
        dni = ficheiro.readUTF();
        if (procura.equalsIgnoreCase(dni)) {
            //Colócase o punteiro ao comezo da liña
            ficheiro.seek(pos * TAM_LINHA);
            nome = ficheiro.readUTF();
            dni = ficheiro.readUTF();
            idade = ficheiro.readInt();
            nota = ficheiro.readFloat();
            repetidor = ficheiro.readBoolean();
            System.out.printf("\t%s\t%s\t%s\t%5d\t%4.2f\t%s\n",
                nome, dni, idade, nota, repetidor);

            System.out.println("Opcións a modificar:");
            System.out.println("\t1. Nome.");
            System.out.println("\t2. DNI.");
            System.out.println("\t3. Idade.");
            System.out.println("\t4. Nota.");
            System.out.println("\t5. Repite.");
            System.out.println("\t6. Ningunha.");
            opcion = validarOpcion(1, 6);
            //Para cada campo volve a colocarse o punteiro
            //na posición que lle corresponda a cada campo
            //xa que despois da lectura anterior, o punteiro
            //atópase ao final da liña
            switch (opcion) {
                case 1: //Nome
                    ficheiro.seek(pos * TAM_LINHA);
                    nome = lerString("\t   Nome (Máximo"
                        + " 10 caracteres): ") + " ";
                    ficheiro.writeUTF(nome.substring(0, 10));
                    break;
                case 2: // DNI
                    ficheiro.seek((pos * TAM_LINHA) + 12);
                    dni = lerString("\t   Dni: ");
                    while (!comprobarDNI(dni)) {
                        System.out.println("\tERRO: o dni "

```

```

        + "é incorrecto."
        + "Volva a introducilo.");
        dni = lerString("\tDNI: ");
    }
    ficheiro.writeUTF(dni);
    break;
case 3: //Idade
    ficheiro.seek((pos * TAM_LINHA) + 23);
    idade = lerInt("\t Idade: ");
    ficheiro.writeInt(idade);
    break;
case 4: // Nota
    ficheiro.seek((pos * TAM_LINHA) + 27);
    nota = lerFloat("\t Nota: ");
    ficheiro.writeFloat(nota);

    break;
case 5: //Repite
    ficheiro.seek((pos * TAM_LINHA) + 31);
    repite = lerByte("\t Repite "
        + "[0-Non 1-Si]: ");
    while (repite != 0 && repite != 1) {
        System.out.println("\tERRO: só "
            + "opción 0 ou 1."
            + "Volva a introducir "
            + "[0-Non 1-Si]:");
        repite = lerByte("\t Repite "
            + "[0-Non 1-Si]: ");
    }
    if (repite == 0) {
        ficheiro.writeBoolean(false);
    } else {
        ficheiro.writeBoolean(true);
    }
    break;
    }
    atopado = true;
} else {
    pos++;
    //Colocamos o punteiro na liña seguinte
    //na posición onde se atopa o valor do dni
    ficheiro.seek((pos*TAM_LINHA) + 12);
}
}
} catch (EOFException eofe) {
    System.out.println("\tFin do ficheiro.");
}
break;
case 3: //Listado alumnado repetidor
try {
    //Colócase o punteiro onde se atopa o 1º valor booleano
    ficheiro.seek(TAM_LINHA-1);
    System.out.println("\t == LISTADO ALUMNADO REPETIDOR ==");
    System.out.printf("\t%s\t%s\t%s\t%s\t%s\t%s\n",
        " NOME ", " DNI ", "IDADE", "NOTA",
        "REPITE");
    while (true) {
        repetidor = ficheiro.readBoolean();
        if (repetidor) {
            ficheiro.seek(ficheiro.getFilePointer()
                - TAM_LINHA);
            nome = ficheiro.readUTF();
            dni = ficheiro.readUTF();
            idade = ficheiro.readInt();
            nota = ficheiro.readFloat();
            System.out.printf("\t%s\t%s\t%s\t%s\t%.2f\t%s\n",
                nome, dni, idade, nota, repetidor);
            ficheiro.seek(ficheiro.getFilePointer()
                + TAM_LINHA);
        } else {
            ficheiro.seek(ficheiro.getFilePointer()
                + TAM_LINHA - 1);
        }
    }
} catch (EOFException eofe) {
    System.out.println("\tFin do ficheiro.");
}
break;

```

```

        case 4: //Listado completo
        try {
            ficheiro.seek(0);
            System.out.println("\t == LISTADO COMPLETO ==");
            System.out.printf("\t%s\t%s\t%s\t%s\t%s\n",
                "NOME", "DNI", "IDADE", "NOTA",
                "REPITE");
            while (true) {
                nome = ficheiro.readUTF();
                dni = ficheiro.readUTF();
                idade = ficheiro.readInt();
                nota = ficheiro.readFloat();
                repetidor = ficheiro.readBoolean();
                System.out.printf("\t%s\t%s\t%5d\t%4.2f\t%s\n",
                    nome, dni, idade, nota, repetidor);
            }
        } catch (EOFException eofe) {
            System.out.println("\tFin do ficheiro.");
        }
        break;
        case 5:
            sair = true;
            break;
    }
} while (!sair);

} catch (FileNotFoundException fnfe) {
    System.out.println("ERRO: O arquivo indicado pode non ser un ficheiro.");
} catch (IOException ioe) {
    System.out.println("ERRO: Houbo un erro de entrada/saída.");
}
}

public static byte validarOpcion(int min, int max) {
    byte opcion = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);
    do {
        repetir = false;
        System.out.print("Seleccione unha opción: ");
        if (sc.hasNextByte()) {
            opcion = sc.nextByte();
            if (opcion < min || opcion > max) {
                System.out.println("\tATENCIÓN! Opción non dispoñible. "
                    + "Volva a seleccionar. ");
                repetir = true;
            }
        } else {
            System.out.println("\tERRO: Débese introducir unha entre "
                + min + " e " + max);
            sc.next();
            repetir = true;
        }
    } while (repetir);
    return opcion;
}

public static String lerString(String cadea) {
    String valor = "";
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextLine()) {
            valor = sc.nextLine();
            repetir = (valor.isEmpty());
        } else {
            sc.next();
        }
    }
    if (repetir) {
        System.out.print("\tERRO: debe introducir algún texto. "
            + "\n\t\tVolva a introducir o valor: ");
    }
}
return valor;

```

```

}

public static int lerInt(String cadea) {
    int valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextInt()) {
            valor = sc.nextInt();
            repetir = (valor < 1);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor numérico. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
    return valor;
}

public static float lerFloat(String cadea) {
    float valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextFloat()) {
            valor = sc.nextFloat();
            repetir = (valor < 1);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor float. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
    return valor;
}

public static byte lerByte(String cadea) {
    byte valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextFloat()) {
            valor = sc.nextByte();
            repetir = (valor < 0);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor numérico. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
    return valor;
}

public static boolean comprobarDNI (String num) {
    boolean correcto = false;
    if (num.length() == 9) {
        String letra = num.substring(8);
        num = num.substring(0,8);
        if ((num.matches("\\d{8}")
            && (letra.matches("[a-zA-Z]{1}")))) {
            int i = Integer.parseInt(num);
            char a = letra.charAt(0);
            if (validarDNI(i,a)) {

```

```

        correcto = true;
    }
}
return correcto;
}
public static boolean validarDNI(int num, char letra) {
    char letras[] = {'T','R','W','A','G','M','Y','F','P','D','X','B','N','J',
        'Z','S','Q','V','H','L','C','K','E'};
    int resto = num % 23;
    return letras[resto] == Character.toUpperCase(letra);
}
}

```

As imaxes que se mostran a continuación mostran partes da execución da aplicación.

- Introducción dun alumno novo no ficheiro:

```

run:
MENÚ

1. Engadir alumno.
2. Modificar datos alumno.
3. Listado alumnado repetidor.
4. Listado completo.
5. Saír.

Seleccione unha opción: 1
== NOVO ESTUDIANTE ==
Nome (Máximo 10 caracteres): Ana
Dni: 11111111H
Idade: 23
Nota: 5,6
Repite [0-Non 1-Sí]: 1

MENÚ
1. Engadir alumno.

```

- Modificar datos dun alumno:

```

run:
MENÚ

1. Engadir alumno.
2. Modificar datos alumno.
3. Listado alumnado repetidor.
4. Listado completo.
5. Saír.

Seleccione unha opción: 2
Introduce o DNI do alumno a procurar: 22222222j
== RESULTADO ==
NOME          DNI          IDADE  NOTA  REPITE
Pedro          22222222J      22    9,70   false

Opcións a modificar:
1. Nome.
2. DNI.
3. Idade.
4. Nota.
5. Repite.
6. Ningunha.

Seleccione unha opción: 4
Nota: 9,95

```

Se a continuación se listaran todos os alumnos, o alumno Pedro aparecería coa nova nota.

- Listado do alumnado que é repetidor:


```

MENÚ
1. Engadir alumno.
2. Modificar datos alumno.
3. Listado alumnado repetidor.
4. Listado completo.
5. Saír.
Seleccione unha opción: 3
== LISTADO ALUMNADO REPETIDOR ==
      NOME          DNI          IDADE  NOTA  REPITE
Ana          11111111H          23   5,60    true
Fin do ficheiro.
MENÚ

```

Como se mostra na imaxe, só hai no ficheiro un alumno que repite.

- Listado completo de todos os alumnos:

```

MENÚ
1. Engadir alumno.
2. Modificar datos alumno.
3. Listado alumnado repetidor.
4. Listado completo.
5. Saír.
Seleccione unha opción: 4
== LISTADO COMPLETO ==
      NOME          DNI          IDADE  NOTA  REPITE
Ana          11111111H          23   5,60    true
Pedro        22222222J          22   9,95    false
Fin do ficheiro.
MENÚ

```

Neste caso só hai dous alumnos almacenados no ficheiro.

1.3.6 Tarefa 6. Serialización de obxectos

Enunciado

Deberase modificar a tarefa 5 de forma que os alumnos sexan obxectos de tipo Alumno. Para esta tarefa só se realizarán dúas das operacións da tarefa 5:

- Engadir novos alumnos: permitirá introducir os datos dun alumno novo. Este alumno gardarase en forma de obxecto en memoria. Para elo, deberase utilizar a estrutura de almacenamento que mellor se axuste a esta situación.
- Saír: ademais de saír da aplicación, almacenará os datos de todos os obxectos de tipo alumno nun ficheiro.

Solución

O código da clase Alumno é o seguinte:

```

package ud05_a02_tarefa6;
import java.io.Serializable;
public class Alumno implements Serializable{
    String nome;
    String dni;
    int idade;
    float nota;
    boolean repite;

    Alumno(String n, String d, int i, float cal, boolean r) {
        if (n.length() > 10) {
            nome = n.substring(0, 10);

```

```

    } else {
        nome = n;
    }

    dni = d;
    idade = i;
    nota = cal;
    repite = r;
}

@Override
public String toString() {
    return nome+"\t"+dni+"\t"+idade+"\t"+nota+"\t"+repite;
}
}

```

O código da classe principal é o seguinte:

```

package ud05_a02_tarefa6;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.Scanner;
public class UD05_A02_Tarefa6 {
    public static void main(String[] args) {
        byte opcion = 0;
        boolean sair = false;
        String nome;
        String dni;
        int idade;
        float nota;
        byte repite;
        boolean repetidor;
        ArrayList<Alumno> alumnado = new ArrayList<>();

        do {
            System.out.println("MENÚ");
            System.out.println("\t1. Engadir alumno.");
            System.out.println("\t2. Sair.");
            opcion = validarOpcion(1, 2);
            switch (opcion) {
                case 1: //Engadir alumno
                    System.out.println("\t == NOVO ESTUDIANTE ==");
                    nome = lerString("\t Nome (Máximo 10 caracteres): ");
                    dni = lerString("\t Dni: ");
                    while (!comprobarDNI(dni)) {
                        System.out.println("\tERRO: o dni é incorrecto."
                            + "Volva a introducilo.");
                        dni = lerString("\tDNI: ");
                    }
                    idade = lerInt("\t Idade: ");
                    nota = lerFloat("\t Nota: ");
                    repite = lerByte("\t Repite [0-Non 1-Si]: ");
                    while (repite != 0 && repite != 1) {
                        System.out.println("\tERRO: só opción 0 ou 1."
                            + "Volva a introducir [0-Non 1-Si]:");
                        repite = lerByte("\t Repite [0-Non 1-Si]: ");
                    }
                    repetidor = (repite != 0);
                    alumnado.add(new Alumno(nome, dni, idade, nota, repetidor));
                    break;
                case 2:
                    boolean rdo = serializar(alumnado);
                    if (rdo) {
                        System.out.println("A escritura no ficheiro foi exitosa.");
                    } else {
                        System.out.println("AVISO: Houbo un erro, non se puido "
                            + "escribir correctamente no ficheiro.");
                    }
                    sair = true;
                    break;
            }
        } while (!sair);

        public static boolean serializar(ArrayList<Alumno> alumnado) {
            boolean correcto = true;

```

```

try (ObjectOutputStream ficheiro =
    new ObjectOutputStream(new FileOutputStream("alumnos.dat"))){
    Iterator<Alumno> iterador = alumnado.iterator();
    while(iterador.hasNext()){
        Alumno alumno = iterador.next();
        ficheiro.writeObject(alumno);
    }
} catch (IOException e) {
    System.out.println("ERRO: Problemas de entrada/saída");
    correcto = false;
}
return correcto;
}

public static byte validarOpcion(int min, int max) {
    byte opcion = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);
    do {
        repetir = false;
        System.out.print ("Seleccione unha opción: ");
        if (sc.hasNextByte()) {
            opcion = sc.nextByte();
            if (opcion < min || opcion > max) {
                System.out.println("\tATENCIÓN! Opción non dispoñible. "
                    + "Volva a seleccionar. ");
                repetir = true;
            }
        } else {
            System.out.println("\tERRO: Débese introducir unha entre "
                + min + " e " + max);
            sc.next();
            repetir = true;
        }
    } while (repetir);
    return opcion;
}

public static String lerString(String cadea) {
    String valor = "";
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextLine()) {
            valor = sc.nextLine();
            repetir = (valor.isEmpty());
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir algún texto. "
                + "\n\t\tVolva a introducir o valor: ");
        }
    }
    return valor;
}

public static int lerInt(String cadea) {
    int valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextInt()) {
            valor = sc.nextInt();
            repetir = (valor < 1);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor numérico. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
}

```

```

        return valor;
    }

    public static float lerFloat(String cadea) {
        float valor = 0;
        boolean repetir;
        Scanner sc = new Scanner(System.in);

        System.out.print(cadea);
        repetir = true;
        while (repetir) {
            if (sc.hasNextFloat()) {
                valor = sc.nextFloat();
                repetir = (valor < 1);
            } else {
                sc.next();
            }
            if (repetir) {
                System.out.print("\tERRO: debe introducir un valor float. "
                    + "\n\t\tVolva a introducir un número: ");
            }
        }
        return valor;
    }

    public static byte lerByte(String cadea) {
        byte valor = 0;
        boolean repetir;
        Scanner sc = new Scanner(System.in);

        System.out.print(cadea);
        repetir = true;
        while (repetir) {
            if (sc.hasNextFloat()) {
                valor = sc.nextByte();
                repetir = (valor < 0);
            } else {
                sc.next();
            }
            if (repetir) {
                System.out.print("\tERRO: debe introducir un valor numérico. "
                    + "\n\t\tVolva a introducir un número: ");
            }
        }
        return valor;
    }

    public static boolean comprobarDNI (String num) {
        boolean correcto = false;
        if (num.length() == 9){
            String letra = num.substring(8);
            num = num.substring(0,8);
            if ((num.matches("\\d{8}")
                && (letra.matches("[a-zA-Z]{1}")))) {
                int i = Integer.parseInt(num);
                char a = letra.charAt(0);
                if (validarDNI(i,a)) {
                    correcto = true;
                }
            }
        }
        return correcto;
    }

    public static boolean validarDNI(int num, char letra) {
        char letras[] = {'T','R','W','A','G','M','Y','F','P','D','X','B','N','J',
            'Z','S','Q','V','H','L','C','K','E'};
        int resto = num % 23;
        return letras[resto] == Character.toUpperCase(letra);
    }
}

```

1.3.7 Tarefa 7. Lectura e modificación de obxectos almacenados nun ficheiro

Enunciado

Deberase modificar a tarefa 6 para que realice exactamente as mesmas operacións que a tarefa 5. As diferenzas que se deben ter en conta son as seguintes:

- Ao comezo da aplicación deberase ler o ficheiro de obxectos e cargalos en memoria na estrutura de almacenamento xa utilizada na tarefa 6.
- A información de cada alumno xestionarase a través de obxectos de tipo Alumno.
- Ao saír da aplicación, tal e como se indicou na tarefa 6, deberanse gardar no ficheiro todos os obxectos de tipo Alumno que están en memoria almacenados na estrutura de almacenamento elixida.

Solución

O código da clase Alumno xa non se inclúe, pois é o mesmo que o da tarefa 6.

O código da clase principal é o seguinte:

```
package ud05_a02_tarefa7;
import java.io.EOFException;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.Scanner;
public class U05_A02_Tarefa7 {
    public static void main(String[] args) {
        Alumno alumno;
        byte opcion = 0;
        boolean sair = false;
        String nome;
        String dni;
        int idade;
        float nota;
        byte repite;
        boolean repetidor;
        ArrayList<Alumno> alumnado = new ArrayList<>();
        Iterator<Alumno> iterador;

        lerFicheiro(alumnado);
        do {
            System.out.println("MENÚ");
            System.out.println("\t1. Engadir alumno.");
            System.out.println("\t2. Modificar datos alumno.");
            System.out.println("\t3. Listado alumnado repetidor.");
            System.out.println("\t4. Listado completo.");
            System.out.println("\t5. Sair.");
            opcion = validarOpcion(1, 5);
            switch (opcion) {
                case 1: //Engadir alumno
                    System.out.println("\t == NOVO ESTUDIANTE ==");
                    nome = lerString("\t Nome (Máximo 10 caracteres): ");
                    while (nome.length() > 10) {
                        System.out.println("\tERRO: o nome é incorrecto.");
                        nome = lerString("\tNome: ");
                    }
                    dni = lerString("\t Dni: ");
                    while (!comprobarDNI(dni)) {
                        System.out.println("\tERRO: o dni é incorrecto.");
                        dni = lerString("\tDNI: ");
                    }
                    idade = lerInt("\t Idade: ");
                    nota = lerFloat("\t Nota: ");
                    repite = lerByte("\t Repite [0-Non 1-Si]: ");
                    while (repite != 0 && repite != 1) {
                        System.out.println("\tERRO: só opción 0 ou 1.");
                        repite = lerByte("\t Repite [0-Non 1-Si]: ");
                    }
                    alumno = new Alumno(nome, dni, idade, nota, repite);
                    alumnado.add(alumno);
                    iterador = alumnado.iterator();
                    break;
                case 2: //Modificar datos alumno
                    iterador = alumnado.iterator();
                    while (iterador.hasNext()) {
                        alumno = iterador.next();
                        System.out.println("\t1. Modificar datos alumno.");
                        System.out.println("\t2. Listado alumnado repetidor.");
                        System.out.println("\t3. Listado completo.");
                        System.out.println("\t4. Sair.");
                        opcion = validarOpcion(1, 5);
                        switch (opcion) {
                            case 1: //Modificar datos alumno
                                nome = lerString("\t Nome (Máximo 10 caracteres): ");
                                while (nome.length() > 10) {
                                    System.out.println("\tERRO: o nome é incorrecto.");
                                    nome = lerString("\tNome: ");
                                }
                                dni = lerString("\t Dni: ");
                                while (!comprobarDNI(dni)) {
                                    System.out.println("\tERRO: o dni é incorrecto.");
                                    dni = lerString("\tDNI: ");
                                }
                                idade = lerInt("\t Idade: ");
                                nota = lerFloat("\t Nota: ");
                                repite = lerByte("\t Repite [0-Non 1-Si]: ");
                                while (repite != 0 && repite != 1) {
                                    System.out.println("\tERRO: só opción 0 ou 1.");
                                    repite = lerByte("\t Repite [0-Non 1-Si]: ");
                                }
                                alumno.setNome(nome);
                                alumno.setDni(dni);
                                alumno.setIdade(idade);
                                alumno.setNota(nota);
                                alumno.setRepite(repite);
                                alumnado.set(alumnado.indexOf(alumno), alumno);
                                iterador = alumnado.iterator();
                                break;
                            case 2: //Listado alumnado repetidor
                                System.out.println("\t1. Listado alumnado repetidor.");
                                System.out.println("\t2. Listado completo.");
                                System.out.println("\t3. Sair.");
                                opcion = validarOpcion(1, 5);
                                break;
                            case 3: //Listado completo
                                System.out.println("\t1. Listado alumnado repetidor.");
                                System.out.println("\t2. Listado completo.");
                                System.out.println("\t3. Sair.");
                                opcion = validarOpcion(1, 5);
                                break;
                            case 4: //Sair
                                sair = true;
                                break;
                        }
                    }
                    break;
                case 3: //Listado alumnado repetidor
                    System.out.println("\t1. Listado alumnado repetidor.");
                    System.out.println("\t2. Listado completo.");
                    System.out.println("\t3. Sair.");
                    opcion = validarOpcion(1, 5);
                    break;
                case 4: //Listado completo
                    System.out.println("\t1. Listado alumnado repetidor.");
                    System.out.println("\t2. Listado completo.");
                    System.out.println("\t3. Sair.");
                    opcion = validarOpcion(1, 5);
                    break;
                case 5: //Sair
                    sair = true;
                    break;
            }
        } while (!sair);
        guardarFicheiro(alumnado);
    }
}
```

```

        + "Volva a introducir [0-Non 1-Si:]);
        repite = lerByte("\t Repite [0-Non 1-Si: ");
    }
    repetidor = (repite != 0);
    alumnado.add(new Alumno(nome, dni, idade, nota, repetidor));
    break;
case 2: //Modificar datos alumno
    boolean atopado = false;
    String procura = lerString("\t Introduce o DNI do alumno"
        + " a procurar: ");
    while (!comprobarDNI(procura)) {
        System.out.println("\tERRO: o dni é incorrecto."
            + "Volva a introduciilo.");
        procura = lerString("\tDNI: ");
    }
    System.out.println("\t == RESULTADO ==");
    System.out.printf("\t%s\t%s\t%s\t%s\t%s\n", " NOME ",
        " DNI ", "IDADE", "NOTA", "REPITE");
    iterador = alumnado.iterator();
    while (iterador.hasNext() && !atopado) {
        alumno = iterador.next();
        if (alumno.dni.equalsIgnoreCase(procura)) {
            atopado = true;
            System.out.println("\t"+alumno.toString());
            System.out.println("Opcións a modificar:");
            System.out.println("\t1. Nome.");
            System.out.println("\t2. DNI.");
            System.out.println("\t3. Idade.");
            System.out.println("\t4. Nota.");
            System.out.println("\t5. Repite.");
            System.out.println("\t6. Ningunha.");
            opcion = validarOpcion(1, 6);
            switch (opcion) {
                case 1: //Nome
                    nome = lerString("\t Nome (Máximo"
                        + " 10 caracteres): ") + " ";
                    alumno.nome = nome.substring(0,10);
                    break;
                case 2: // DNI
                    dni = lerString("\t Dni: ");
                    while (!comprobarDNI(dni)) {
                        System.out.println("\tERRO: o dni "
                            + "é incorrecto."
                            + "Volva a introduciilo.");
                        dni = lerString("\tDNI: ");
                    }
                    alumno.dni = dni;
                    break;
                case 3: //Idade
                    idade = lerInt("\t Idade: ");
                    alumno.idade = idade;
                    break;
                case 4: // Nota
                    nota = lerFloat("\t Nota: ");
                    alumno.nota = nota;
                    break;
                case 5: //Repite
                    repite = lerByte("\t Repite "
                        + "[0-Non 1-Si: ");
                    while (repite != 0 && repite != 1) {
                        System.out.println("\tERRO: só "
                            + "opción 0 ou 1."
                            + "Volva a introducir "
                            + "[0-Non 1-Si:");
                        repite = lerByte("\t Repite "
                            + "[0-Non 1-Si: ");
                    }
                    alumno.repite = (repite != 0);
                    break;
            }
        }
    }
    if (!atopado) {
        System.out.println("Non se atopou o alumno co dni "+procura);
    }
    break;
case 3: //Listado alumnado repetidor
    System.out.println("\t == LISTADO ALUMNADO REPETIDOR ==");
    System.out.printf("\t%s\t%s\t%s\t%s\t%s\n",
        " NOME ", " DNI ", "IDADE", "NOTA",

```

```

        "REPITE");
        iterador = alumnado.iterator();
        while (iterador.hasNext()) {
            alumno = iterador.next();
            if (alumno.repite) {
                System.out.println("\t" + alumno.toString());
            }
        }
        break;
    case 4: //Listado completo
        System.out.println("\t == LISTADO COMPLETO ==");
        System.out.printf("\t%s\t%s\t%s\t%s\t%s\n",
            "NOME", "DNI", "IDADE", "NOTA",
            "REPITE");
        iterador = alumnado.iterator();
        while (iterador.hasNext()) {
            alumno = iterador.next();
            System.out.println("\t" + alumno.toString());
        }
        break;
    case 5:
        boolean rdo = serializar(alumnado);
        if (rdo) {
            System.out.println("A escritura no ficheiro foi exitosa.");
        } else {
            System.out.println("AVISO: Houbo un erro, non se puido "
                + "escribir correctamente no ficheiro.");
        }
        sair = true;
        break;
    }
} while (!sair);
}

public static boolean lerFicheiro(ArrayList<Alumno> alumnado) {
    boolean correcto = true;
    Alumno a;
    try (ObjectInputStream fichero =
        new ObjectInputStream(new FileInputStream("alumnos.dat"))){
        while (true) {
            alumnado.add((Alumno) fichero.readObject());
        }
    } catch (EOFException e) {
        //Non se pon nada para que non salga ningunha mensaxe por pantalla
    } catch (ClassNotFoundException e) {
        System.out.println("ERRO: Non se pode atopar a clase da serialización");
        correcto = false;
    } catch (IOException e) {
        System.out.println("AVISO: Problemas de entrada/saída. "
            + "Pode que o ficheiro aínda non exista.");
        correcto = false;
    }
    return correcto;
}

public static boolean serializar(ArrayList<Alumno> alumnado) {
    boolean correcto = true;
    try (ObjectOutputStream fichero =
        new ObjectOutputStream(new FileOutputStream("alumnos.dat"))){
        Iterator<Alumno> iterador = alumnado.iterator();
        while (iterador.hasNext()) {
            Alumno alumno = iterador.next();
            fichero.writeObject(alumno);
        }
    } catch (IOException e) {
        System.out.println("ERRO: Problemas de entrada/saída");
        correcto = false;
    }
    return correcto;
}

public static byte validarOpcion(int min, int max) {
    byte opcion = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);
    do {
        repetir = false;
        System.out.print ("Seleccione unha opción: ");

```

```

        if (sc.hasNextByte()) {
            opcion = sc.nextByte();
            if (opcion < min || opcion > max) {
                System.out.println("\tATENCIÓN! Opción non dispoñible. "
                    + "Volva a seleccionar. ");
                repetir = true;
            }
        } else {
            System.out.println("\tERRO: Débese introducir unha entre "
                + min + " e " + max);
            sc.next();
            repetir = true;
        }
    } while (repetir);
    return opcion;
}

public static String lerString(String cadea) {
    String valor = "";
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextLine()) {
            valor = sc.nextLine();
            repetir = (valor.isEmpty());
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir algún texto. "
                + "\n\t\tVolva a introducir o valor: ");
        }
    }
    return valor;
}

public static int lerInt(String cadea) {
    int valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextInt()) {
            valor = sc.nextInt();
            repetir = (valor < 1);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor numérico. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
    return valor;
}

public static float lerFloat(String cadea) {
    float valor = 0;
    boolean repetir;
    Scanner sc = new Scanner(System.in);

    System.out.print(cadea);
    repetir = true;
    while (repetir) {
        if (sc.hasNextFloat()) {
            valor = sc.nextFloat();
            repetir = (valor < 1);
        } else {
            sc.next();
        }
        if (repetir) {
            System.out.print("\tERRO: debe introducir un valor float. "
                + "\n\t\tVolva a introducir un número: ");
        }
    }
}

```



```

        return valor;
    }

    public static byte lerByte(String cadea) {
        byte valor = 0;
        boolean repetir;
        Scanner sc = new Scanner(System.in);

        System.out.print(cadea);
        repetir = true;
        while (repetir) {
            if (sc.hasNextFloat()) {
                valor = sc.nextByte();
                repetir = (valor < 0);
            } else {
                sc.next();
            }
            if (repetir) {
                System.out.print("\tERRO: debe introducir un valor numérico. "
                    + "\n\t\tVolva a introducir un número: ");
            }
        }
        return valor;
    }

    public static boolean comprobarDNI (String num) {
        boolean correcto = false;
        if (num.length() == 9){
            String letra = num.substring(8);
            num = num.substring(0,8);
            if ((num.matches("\\d{8}"))
                && (letra.matches("[a-zA-Z]{1}"))) {
                int i = Integer.parseInt(num);
                char a = letra.charAt(0);
                if (validarDNI(i,a)) {
                    correcto = true;
                }
            }
        }
        return correcto;
    }

    public static boolean validarDNI(int num, char letra) {
        char letras[] = {'T','R','W','A','G','M','Y','F','P','D','X','B','N','J',
            'Z','S','Q','V','H','L','C','K','E'};
        int resto = num % 23;
        return letras[resto] == Character.toUpperCase(letra);
    }
}

```

2. Autoavaliación

▪	▪	▪	
▪	▪	▪	

2.1 Enunciado

Deberase crear unha aplicación que permita almacenar e recuperar a información dos historiais médicos dun hospital. A información que nos interesa de cada historial é a seguinte:

- N° historial: que será un número de historial correlativo.
- N° SS do paciente.
- Idade do paciente.
- Presión sanguínea máxima.

- Presión sanguínea mínima.
- Grupo sanguíneo: que poderá ser A, B, 0, ou AB. Se non se indica correctamente un destes valores, o valor por defecto será 0.
- Factor RH: + ou -. Se non se indica correctamente un destes valores, o valor por defecto será -.

A aplicación deberá permitir a introdución de novos historiais médicos e almacenalos en tres tipos de ficheiros:

- Ficheiro de texto.
- Ficheiro de acceso aleatorio.
- Ficheiro de obxectos.

Mentres se estea a traballar co programa, os datos dos historiais manteranse en memoria e unha vez que se peche a aplicación deberán escribirse todos os datos nos tres tipos de ficheiros anteriores.

Para poder cargar os datos en memoria é posible facelo desde calquera dos tres tipos de ficheiros, xa que a información que estes conteñen debe ser idéntica. Por elo, deberá permitírselle ao usuario indicar de cal dos tres tipos de ficheiros se lerá a información.

Habrá que ter en conta que a primeira vez que se executa o programa non existirán os ficheiros.

En todo momento, deberase manter informado ao usuario, por exemplo de calquera erro na apertura ou peche de ficheiros, se non hai historiais a listar, etc.

O programa deberá manterse libre de posibles erros, como por exemplo de que o usuario introduza un valor non apropiado.

Unha vez seleccionado o tipo de ficheiro desde o que se lerán os datos, o usuario poderá realizar as seguintes operacións:

- Engadir un novo historial: que engadirá en memoria un novo historial.
- Listar todos os historias: deberase mostrar toda a información dos historiais que existan ata o momento. A saída deberá estar formatada.
- Saír: que ademais de saír, volcará toda a información que haxa na estrutura de almacenamento de memoria aos tres tipos de ficheiros.

Exemplo de execución

Na seguinte imaxe pode verse o menú inicial que debería saír para indicar cal dos tres tipos de ficheiros se vai a abrir. Como é a primeira vez que se executa o programa infórmase de que se vai a proceder a crear e abrir o ficheiro. Tamén se ve o menú que debería aparecer unha vez aberto o ficheiro. Se seleccionamos a opción de listar o contido deste menú, aparece unha mensaxe indicando que aínda non hai ningún historial, pois se acaba de crear o ficheiro por primeira vez.

```

run:
TIPOS DE FICHEIROS
    1. Ficheiro de texto.
    2. Ficheiro aleatorio.
    3. Ficheiro de obxectos.
Seleccione unha opción: 1
Creando ficheiro....
Abrindo ficheiro....
MENÚ
    1. Engadir historial.
    2. Listado completo.
    3. Saír.
Seleccione unha opción: 2
    == LISTADO HISTORIAS ==
AVISO: Aínda non hai ningún historial.

MENÚ

```

Nesta outra imaxe, podemos ver como se engade un novo historial e como se mostra o listado de historiais existentes unha vez inserido o historial.

```

MENÚ
    1. Engadir historial.
    2. Listado completo.
    3. Saír.
Seleccione unha opción: 1
    == NOVO HISTORIAL ==
        Núm. Historial: 1
        Núm. SS: 11111
        Idade: 56
        Presión Máxima: 10,6
        Presión Mínima: 8,6
        Grupo Sanguíneo: A
        Factor RH: +
MENÚ
    1. Engadir historial.
    2. Listado completo.
    3. Saír.
Seleccione unha opción: 2
    == LISTADO HISTORIAS ==
        NUMERO      N° SS      IDADE      PRESIÓN MÁX      PRESIÓN MÍN      GRUPO SANG      FACTOR RH
        1           11111      56         10.6             8.6             A               +
MENÚ

```

Na seguinte imaxe, vese o resultado de saír do programa, no que se indica que se escribi-ron todos os historiais nos tres ficheiros.

```

MENÚ
    1. Engadir historial.
    2. Listado completo.
    3. Saír.
Seleccione unha opción: 3
A escritura nos ficheiros foi exitosa.
BUILD SUCCESSFUL (total time: 2 minutes 10 seconds)

```

O resultado de pechar o programa, vese nas dúas imaxes seguintes. Na primeira imaxe ve-ses que se crearon os tres ficheiros no directorio do proxecto.

```

historiaisObxectos.dat
historiaisAleatorios.dat
historiaisTexto.txt
build.xml
manifest.mf
build
test
src
nbproject

```

Nesta segunda imaxe vese o contido de cada un dos ficheiros. Neste caso, só conteñen o único historial que se engadiu a través da aplicación.

